分 类 号_____ 学号_____
学校代码_____ 密级_____

華中科技大學

硕士学位论文

(学术型□ 专业型☑)

基于驱动程序替换的物联网设备固件仿真技术研究

学位申请人：

学 科 专 业：

指 导 教 师：

答 辩 日 期：

A Dissertation Submitted in Partial Fulfillment of the Requirements for

Research on IoT Device Firmware Emulation Technology Based on Driver Replacement

Candidate :

Major :

Supervisor :

Huazhong University of Science and Technology

Wuhan 430074, P. R. China

March, 2026

独创性声明

本人声明所呈交的学位论文是我个人在导师的指导下进行的研究工作及取得的研究成果。尽我所知，除文中已标明引用的内容外，本论文不包含任何其他人或集体已经发表或撰写过的研究成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

日期：　　年　　月　　日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权华中科技大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本论文属于 保　密 □，在 ____ 年解密后适用本授权书。
不保密 □。

（请在以上方框内打“√”）

学位论文作者签名：　　指导教师签名：

日期：　　年　　月　　日　　日期：　　年　　月　　日

摘　　要

随着物联网技术的快速发展，物联网设备数量呈现爆炸式增长，其固件安全问题日益突出。固件仿真技术是进行固件安全测试和漏洞挖掘的重要手段，然而现有的固件仿真方法存在外设建模成本高、硬件依赖强、自动化程度不足等问题。传统的基于硬件建模的方法需要为每种外设手动建立仿真模型，工作量大且难以覆盖多样化的硬件生态；基于驱动层替换的方法虽然避免了底层硬件建模，但驱动函数的定位与替换仍高度依赖人工干预，难以实现规模化应用。

针对上述问题，本文提出一种结合大语言模型的自动化固件仿真方法。该方法的核心思想是通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型的代码理解与生成能力自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量，从而在无需手动建立外设模型的情况下实现固件在仿真环境中的稳定运行。

本文的主要工作包括三个方面。首先，设计并实现了面向嵌入式固件的硬件依赖代码自动识别模块，该模块通过 MMIO 模式识别、数据流追踪与递归归纳技术，从固件中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界，构建可查询的驱动知识库。其次，构建了基于大语言模型的驱动函数分析与替换模块，该模块利用大语言模型理解驱动函数的语义，通过结构化提示词模板和函数分类策略，自动生成可在仿真环境中运行的替代代码，并通过编译修复循环保障代码的可编译性。最后，引入了动态测试反馈闭环机制，当固件在仿真环境中运行出现异常时，系统能够自动分析异常原因并反馈给驱动替换模块，通过”生成—测试—反馈—修正”的闭环流程持续优化仿真质量。

实验结果表明，本文方法能够有效支持多种嵌入式平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）的固件仿真，显著降低了固件仿真的技术门槛和人工成本，为后续的模糊测试和漏洞挖掘奠定了基础。

关键词：物联网安全；固件仿真；驱动程序替换；大语言模型；静态分析

Abstract

With the rapid development of Internet of Things (IoT) technology, the number of IoT devices has grown explosively, and firmware security issues have become increasingly prominent. Firmware emulation is an important approach for firmware security testing and vulnerability discovery. However, existing firmware emulation methods face challenges such as high peripheral modeling costs, strong hardware dependencies, and insufficient automation. Traditional hardware modeling-based methods require manually building simulation models for each peripheral, which is labor-intensive and difficult to cover the diverse hardware ecosystem. Although driver layer replacement-based methods avoid low-level hardware modeling, the identification and replacement of driver functions still heavily rely on manual intervention, making it difficult to achieve large-scale application.

To address these issues, this paper proposes an automated firmware emulation method combined with large language models (LLMs). The core idea is to automatically identify hardware-dependent code in firmware through static analysis, use the code understanding and generation capabilities of LLMs to automatically generate alternative driver implementations, and build a dynamic testing feedback loop to iteratively optimize emulation quality, thereby enabling firmware to run stably in emulation environments without manually building peripheral models.

The main contributions of this paper include three aspects. First, a hardware dependency code automatic identification module for embedded firmware is designed and implemented, which automatically extracts hardware-related function call points, peripheral register access patterns, and driver function boundaries from firmware through MMIO pattern recognition, data flow tracking, and recursive induction techniques, constructing a queryable driver knowledge base. Second, a driver function analysis and replacement module based on LLMs is built, which utilizes LLMs to understand driver function semantics, automatically

generates alternative code that can run in emulation environments through structured prompt templates and function classification strategies, and ensures code compilability through a compilation repair loop. Finally, a dynamic testing feedback loop mechanism is introduced. When firmware runs abnormally in the emulation environment, the system can automatically analyze the cause of the abnormality and feed it back to the driver replacement module, continuously optimizing emulation quality through a ”generate-test-feedback-revise” closed-loop process.

Experimental results show that the proposed method can effectively support firmware emulation for multiple embedded platforms (STM32, NXP) and operating systems (FreeRTOS, RT-Thread, bare metal), significantly reducing the technical barrier and manual cost of firmware emulation, laying the foundation for subsequent fuzzing and vulnerability discovery.

Keywords: IoT Security; Firmware Emulation; Driver Replacement; Large Language Model; Static Analysis

目　　录

1 绪论

1.1 研究背景与意义

近年来，随着物联网（Internet of Things, IoT）技术迅速发展，物联网设备已被广泛用于智能家居、智慧城市、新能源、车联网、医疗健康等多个应用领域。根据GSMA 发布的《The mobile economy 2020》报告显示，2019 年全球物联网总连接数达到 120 亿，预计到 2025 年，全球物联网总连接数规模将达到 246 亿，年复合增长率高达 13%[?]。物联网操作系统是一种面向物联网设备和应用的轻量级嵌入式操作系统，它可以提供更好的设备管理、数据处理和安全性能。目前，市场上有许多物联网操作系统，例如华为的 OpenHarmony[?]、亚马逊的 FreeRTOS[?]、Linux 基金会的 Zephyr[?] 等。然而，物联网和嵌入式操作系统面临着严峻的安全挑战[?]，如恶意攻击、数据泄露、隐私侵犯等，这些安全问题可能会给用户带来损失和危害。在真实世界中发生的物联网安全问题也层出不穷。2018 年，一款名为”Mirai” 的恶意软件[?] 利用了 IoT 设备的弱点，通过攻击嵌入式设备的操作系统，将其变成僵尸网络的一部分，对互联网上的服务器发起了大规模的 DDoS 攻击。俄乌冲突期间，乌克兰遭遇了俄罗斯发起的破坏性网络攻击，该攻击利用一种名为 Industroyer 的恶意软件控制电力系统中的开关和断路器，意图瘫痪乌克兰的电力系统。这些安全事件的不断发生表明，当前物联网和嵌入式操作系统安全问题的研究刻不容缓。

固件仿真（Firmware Re-Hosting）技术是一种克服固件分析硬件依赖问题的有效方法，它通过在商用计算机和服务器的软件模拟器上运行嵌入式系统固件，实现对嵌入式设备的软件级模拟，使得包含嵌入式系统的固件无需实际硬件即可进行安全测试[?]。然而，当前固件仿真技术的**自动化程度仍然严重不足**，这主要体现在以下几个方面：首先，现有的外设仿真方法大多需要人工编写模型或基于有限规则的自动推断。硬件在环方案需要为每种外设手动配置物理设备连接；基于原始硬件的外设建模方案需要人工分析数据手册并编写仿真代码；基于程序分析的方案虽然能自动推断部分外设行为，但推断规则有限，难以覆盖复杂外设的状态机逻辑。其次，

驱动层替换方案（如 HALucinator）虽然利用了微控制器制造商提供的硬件抽象层（HAL），通过函数 Hook 的方式进行替换，但驱动函数的定位、语义理解和替换代码生成仍然需要大量人工干预。分析人员需要手动识别固件中哪些部分存在硬件依赖并需要替换，理解驱动函数与硬件的交互逻辑，并编写相应的替代代码，这个过程不仅耗时耗力，而且容易出错。这种**自动化程度不足导致的人工成本高昂**，严重制约了固件仿真技术在大规模安全测试中的应用，成为当前物联网设备安全领域亟待解决的关键问题。

现有固件仿真技术的自动化不足主要体现在三个核心环节。在驱动函数定位环节，识别固件中哪些部分存在硬件依赖并需要替换是一项具有挑战性的任务。在嵌入式程序中，硬件依赖代码往往分散在各个模块中，通过函数调用、全局变量访问、寄存器操作等多种形式存在，手动定位这些代码需要分析人员对固件结构和硬件特性有深入理解，过程既耗时又容易出错。在外设建模环节，当前的自动化方法只能处理简单的外设行为，对于网络控制器、USB 设备等具有多通道、DMA 传输、中断嵌套等特性的复杂外设，往往需要人工编写仿真模型。在替换代码生成环节，现有工具主要通过函数 Hook 返回预设值或执行简化逻辑，缺乏对驱动函数内部语义的深入理解，无法准确模拟硬件行为，需要人工编写复杂的替代代码。这三个环节的自动化不足相互叠加，使得固件仿真过程对分析人员经验的依赖程度过高，难以适应大规模、多样化的固件安全测试需求。

针对上述问题，本研究致力于提升固件仿真技术的自动化程度，降低人工成本，实现高效的物联网设备固件安全测试。具体而言，本研究提出一种基于驱动程序替换的固件仿真方法，通过静态分析技术自动识别固件中的硬件依赖代码，利用大语言模型的代码理解能力深入分析驱动函数语义并自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量。该方法的核心创新在于将自动化技术贯穿于驱动函数定位、语义分析和代码生成的全过程，显著减少了对分析人员人工干预的依赖。通过提升固件仿真的自动化水平，本研究不仅能够降低固件安全测试的成本，还能够提高测试效率和覆盖率，为物联网设备的大规模安全测试提供技术支撑，具有重要的理论意义和应用价值。

1.2 国内外研究现状

固件是嵌入式设备中的软件组成部分，通常包含了设备的操作系统、驱动程序、应用程序和配置文件等。固件的安全性对于保护设备和用户数据的完整性和隐私至关重要。然而，由于固件的复杂性、多样性和闭源性，对固件进行有效的安全测试是一项具有挑战性的任务。

基于固件仿真的模糊测试技术在近年来受到了越来越多的关注和研究，出现了一些代表性的工具和系统，如 FIRMADYNE[?]、FUZZWARE[?] 等。这些工具和系统在不同程度上实现了对固件仿真和模糊测试的支持，并在实际的固件中发现了一些新的漏洞。然而，基于固件仿真的模糊测试技术仍然存在一些挑战和问题，如仿真环境的准确性、测试用例的有效性等，需要进一步的改进和优化。下面将从固件仿真技术、嵌入式设备模糊测试技术两个方面阐述当前基于固件仿真的嵌入式设备固件模糊测试技术研究现状。

1.2.1 固件仿真技术研究现状

当前固件仿真的主要挑战是为模拟器中没有实现的硬件提供有效的输入。诸如 QEMU[?] 之类的仿真器提供了模拟各种 CPU 体系结构的能力，但对商用嵌入式系统中使用的各种各样的外设，例如计时器、UART、以太网控制器等，提供的支持却十分有限，而这些外设对于正确实现固件仿真却是十分必要的。

固件仿真技术的发展经历了从依赖真实硬件到全自动化模拟的演进过程。早期研究采用硬件在环（Hardware in the Loop, HIL）的方法，如 Zaddich 等人提出的 Avatar[?] 和 Muench 等人提出的 Avatar2[?]，通过混合执行的方式将固件分析过程中对外设的访问转发到物理硬件中。然而，这种方案仍然依赖实际设备，且仿真环境和物理设备之间的切换影响了运行速度。为摆脱对真实硬件的依赖，研究者提出了多种全自动化仿真方案，主要包括以下三类。

1.2.1.1 基于原始硬件的外设建模方案

解决外设依赖的一种方法是在底层内存映射寄存器接口上实现硬件，或通过观察真实硬件的行为来构建外设模型。然而这是一项费力的任务，扩展性不好。当前

主流的仿真器 QEMU 也仅能仿真 10 余种设备，远不能满足固件仿真工作的需求。

Gustafson 等人提出的 Pretender[?] 工具通过观察固件与物理设备之间真实发生的交互记录，使用机器学习、模式识别等方法为每个外设构建模型，再结合 QEMU 或 angr 等仿真引擎实现固件仿真。这种方法需要真实设备的支持，但在观测到足够的交互模式后，可以自动生成较为准确的外设模型。

1.2.1.2 基于程序分析的外设建模方案

基于程序分析的外设建模方案通过对固件代码进行静态或动态分析，提取固件与外设之间的交互逻辑，构建虚拟外设模型以替代真实硬件。

Feng 等人提出的 P2IM[?] 工具通过动态符号执行和模糊测试，自动识别固件中的输入点和输出点，构建与真实设备相似的虚拟模型。Cao 等人提出的 Laelaps[?] 在因访问未实现的外设而卡住时，切换到符号执行以找到正确输入，引导 QEMU 找到与实际执行最相近的路径。Zhou 等人提出的 uEmu[?] 工具利用符号执行分析固件镜像，将未知的外设寄存器表示为符号，构建推断规则并存储在知识库中。Scharnowski 等人提出的 Fuzzware[?] 使用精确的 MMIO 建模实现有效的固件模糊测试，通过确定固件逻辑如何实际使用硬件生成的值，自动生成模型以提高模糊测试输入数据的有效性。

1.2.1.3 基于系统或驱动层替换的外设仿真方案

一种有前途的方法是基于系统或驱动层替换的仿真技术，或称为高级仿真（High Level Emulation, HLE）技术。它利用固件中的公共抽象来消除为外设提供低级支持的需要。

Chen 等人提出了一个针对 Linux 内核的物联网设备固件仿真平台 Firmadyne[?]，通过修改内核和驱动来添加硬件支持，实现物联网设备的全系统仿真。该工具解决了定制化外围 I/O、非易失性内存、动态文件等仿真技术难题。Li 等人创建了 Para-rehosting[?] 工具，该工具的主要思想是利用 POSIX 接口抽象并实现了一个便携式 MCU，以将嵌入式系统源代码重新编译到商用机器的本机可执行文件中，以便利

用 x86 上许多现有的复杂软件测试工具对代码进行测试。然而，这种方案的实现需要分析师获得固件样本的源代码，而在第三方的固件分析工作中，源代码的获取往往是困难甚至完全不可行的。Clements 等人提出了 HALucinator[?] 工具，该工具利用了微控制器制造商提供的硬件抽象层（Hardware Abstraction Layer, HAL），通过对硬件抽象层函数的抽象与替换进行固件仿真，这项工作在重托管简单的裸金属固件程序时取得了很好的效果。

然而，上述基于系统或驱动层替换的仿真技术自动化程度较差，难以适应大规模安全测试的需要。具体而言，这类技术面临以下两个核心难点：一是难以自动化定位需要进行替换的驱动程序。在嵌入式程序中，识别哪些部分存在硬件依赖并需要替换，往往依赖分析人员的手动干预。手动分析嵌入式程序的复杂性使得这一过程既耗时又容易出错。二是难以自动化生成替换程序。除了识别硬件相关功能，当前的技术也缺乏自动化生成替代程序的能力。分析人员必须手动分析函数的功能，理解其与硬件和中间件的交互，并编写相应的替代代码。这个过程不仅繁琐，而且容易导致人力成本的增加。

1.2.1.4 固件仿真技术对比

表 1.1 对比了当前主流固件仿真技术的优缺点。

在这三类方案中，基于系统或驱动层替换的固件仿真方法是一种有前景的方案。它从固件系统层和驱动函数入手，较好地解决了基于程序分析和原始硬件建模的方法对复杂外设难以完全模拟或建模工作量大等痛点。HALucinator 等工具利用微控制器制造商提供的硬件抽象层（Hardware Abstraction Layer, HAL），通过函数替换的方式实现固件仿真，在重托管简单的裸金属固件程序时取得了很好的效果。

然而，现有驱动层替换方案在自动化方面仍存在明显不足：一方面，驱动函数的定位依赖分析人员的手动干预，识别固件中哪些部分存在硬件依赖并需要替换是一个耗时且容易出错的过程；另一方面，替换程序的生成也缺乏自动化支持，分析人员需要手动分析函数功能、理解其与硬件的交互，并编写相应的替代代码。为解决上述问题，需要设计一种能够自动化定位需要替换的驱动函数、并利用大模型或

表 1.1 当前主流固件仿真技术对比

方案	典型工具	优点	缺点
基于原始硬件的外设建模	QEMU, Pretender	可对外设行为进行精确模拟，提高测试效率和覆盖率	需要为每种外设专门建模，工作量大，通用性差，依赖真实设备观测
基于程序分析的外设建模	P2IM, uEmu, Laelaps, Fuzzware	可根据程序自身信息自动推断外设行为，减少人工干预，提高通用性	难以完全覆盖复杂外设的所有行为，可能存在误报或漏报
基于系统或驱动层替换	HALucinator, Firmadyne, Para-rehosting	在系统层或驱动层对函数进行抽象和替换，降低对硬件的依赖，提高测试速度和灵活性	驱动函数的定位与替换依赖人工干预，自动化程度不足，难以适应大规模安全测试

代码分析技术自动生成替代程序的方案，以提升固件仿真过程的自动化程度，实现高效的物联网设备程序仿真与动态测试。

1.2.2 嵌入式系统模糊测试技术发展现状

嵌入式系统模糊测试技术是一种针对嵌入式设备的软件安全测试方法，它可以自动或半自动地生成随机或变异的输入数据，发送给嵌入式设备的 Web 服务或其他接口，监控设备的运行状态，发现潜在的漏洞或缺陷。目前，嵌入式系统模糊测试技术主要从以下三个方向进行发展：

1.2.2.1 面向真实设备的黑盒模糊测试

直接对嵌入式设备进行模糊测试，不需要对设备进行任何修改或插桩，只需要获取设备的网络接口或串口等信息，然后使用模糊测试工具生成并发送测试用例，监控设备的响应和异常。Chen 等人开发的 IoTFuzzer[?] 工具，就利用了应用程序与物联网设备的交互信息，通过网络接口向嵌入式设备发送模糊测试数据，进行模糊测试工作。这种方法的优点是简单易用，不依赖于设备的内部结构或代码，适用于黑盒测试场景。缺点是测试效率较低，覆盖率较差，难以发现深层次的漏洞。

1.2.2.2 基于固件仿真的灰盒模糊测试

基于固件仿真的模糊测试，通过对嵌入式设备的固件进行逆向工程，提取出可执行的二进制代码，然后使用仿真器在虚拟环境中运行固件，并对其进行模糊测试。基于该方案的仿真工具有 Fuzzware[?]、uEmu[?]、HALucinator[?] 等。这种方法的优点是可以对固件进行插桩或修改，获取更多的运行信息，提高测试效率和覆盖率，发现更多的漏洞。缺点是需要对固件进行复杂的逆向分析，可能遇到加密或保护机制，同时仿真器可能存在不完全或不准确的问题，导致测试结果与真实设备不一致。

1.2.3 大模型程序分析与代码生成技术研究现状

大语言模型（Large Language Model, LLM）在代码理解和生成方面展现出了强大的能力。以 GPT-4[?]、Code Llama[?]、StarCoder[?] 等为代表的代码大模型能够理解复杂的代码逻辑，并生成高质量的代码。近年来，研究者开始探索将 LLM 应用于程序分析和代码生成领域，涌现出多种增强策略以提升模型在复杂编程任务中的表现。

1.2.3.1 面向特定任务的代码生成方法

早期 LLM 通过零样本提示能够生成基础代码，但在处理复杂编程任务时准确性和可靠性仍存在不足。为克服这些局限，研究者发展了以下增强策略：

指令微调（Instruction Tuning）：通过对预训练 LLM 在特定领域数据集上进行精细调整，显著提升模型对特定编程任务的理解和执行能力[?]。DolphCoder 模型通过多样化的指令和自评估机制，结合代码评估目标有效提升了代码生成能力[?]。CodeLSI 框架则通过低秩优化与领域特定指令微调相结合，提高了代码生成的领域特异性和成本效益[?]。

测试驱动的迭代生成：利用测试用例作为反馈信号指导模型逐步修正生成的代码，直至通过所有测试。这种方法通过隐式执行监督信号，结合搜索、重排序或迭代修正等机制动态优化模型输出[?]。RLCF（Reinforcement Learning with Compiler Feedback）方法通过编译器反馈对 LLM 进行额外训练，解决生成代码中可能存在的语言级不变量违规问题[?]。CodeDPO 框架则将偏好学习整合到代码生成过程中，优

化模型优先选择正确且高效的解决方案[?]。

多步推理方法:思维链（Chain-of-Thought, CoT）和程序思维（Program-of-Thought, PoT）等方法通过将复杂问题分解为多个可管理的子步骤，提升模型解决复杂问题的能力[?]。MoTCoder 引入模块化思维指令框架来应对更具挑战性的编程任务[?]。程序思维方法则将中间推理步骤编译为可执行代码片段，实现了" 推理即执行"，大幅提升了数值计算和 API 调用类任务的准确性。

工具增强的大模型：通过集成编译器、调试器、静态分析工具等外部工具，扩展模型的实际应用能力。这种范式将 LLM 定位为"AI 协调者"，通过标准化工具调用协议调度各种工具，形成感知-决策-执行的闭环[?]。MCCoder 系统专注于生成运动控制代码，并结合严格的验证机制以确保代码安全性[?]。RGD 代理调试器利用多 LLM 通过细化和生成指导来自动修复代码中的错误[?]。

1.2.3.2 系统级软件代码生成的挑战与前沿

在系统级软件应用层面，特别是针对物联网设备驱动程序、操作系统内核和嵌入式固件等复杂系统，LLM 的代码生成面临独特挑战。IoT 设备驱动程序开发具有强硬件耦合性、并发中断敏感性、底层资源约束性以及隐式的 DMA/寄存器交互等特征，对 LLM 的代码生成能力提出了更高要求[?]。

当前前沿研究强调构建" 分析—生成—验证" 的闭环系统以应对这些挑战：

静态分析集成：静态分析工具如 CIL、Smatch 和 Frama-C 用于提取接口契约与控制流约束，对代码进行结构化表示[?]。这些工具能够识别不安全的功能、缓冲区溢出和竞态条件等安全漏洞，为 LLM 生成提供结构化上下文。

动态符号执行验证：动态符号执行引擎如 QEMU 和 KLEE 能够生成高覆盖率的路径约束，并反向验证生成代码的内存安全与中断语义正确性[?]。通过结合编译器诊断、安全扫描和符号执行，LLM 可以迭代细化其输出。

端到端流水线：研究者正在构建端到端的" 分析 → 生成 → 符号验证 → 反馈精炼" 流水线[?]。形式化规约被用作生成约束，而验证失败轨迹则被结构化为自然语言反馈以驱动下一轮生成。Agents4PLC 框架通过基于 LLM 的代理自动化了可编程

逻辑控制器（PLC）代码生成与验证，提高了工业控制系统的操作效率和安全性。

这些研究为本文利用 LLM 进行驱动函数分析和替换提供了理论基础，特别是在构建自动化分析-生成-验证闭环方面具有重要的参考价值。

1.2.4 现有研究的局限性分析

综合上述研究现状，现有固件仿真技术主要存在以下局限性：

（1）外设建模依赖人工，难以处理复杂外设行为

现有仿真方法大多依赖人工编写外设模型或基于有限规则的自动推断。硬件在环方案需要为每种外设手动配置物理设备连接；基于原始硬件的外设建模方案需要人工分析数据手册并编写仿真代码；基于程序分析的方案虽然能自动推断部分外设行为，但推断规则有限，难以覆盖复杂外设的状态机逻辑。例如，网络控制器、USB 设备等复杂外设具有多通道、DMA 传输、中断嵌套等特性，现有自动推断方法往往只能处理简单的 MMIO 寄存器读写，无法准确模拟这些复杂交互行为。

（2）缺乏对驱动函数语义的深入理解，仿真结果可能不准确

现有驱动层替换方案（如 HALucinator）主要通过函数 Hook 的方式进行替换，即当固件调用某个 HAL 函数时，返回预设的模拟值或执行简化的模拟逻辑。然而，这种方法缺乏对驱动函数内部语义的深入理解：函数之间的调用关系、参数的数据流传递、外设状态的变化等关键信息往往被忽略。这导致在某些场景下，仿真结果与真实硬件行为存在偏差，可能遗漏重要的程序路径或引入虚假的漏洞。

（3）缺乏自动化反馈机制，难以迭代优化仿真质量

当前大多数仿真工具采用”一次性配置”的模式：分析人员手动配置仿真环境后运行测试，如果发现问题则需要人工介入调试和修复。这种模式缺乏自动化的反馈闭环机制——当仿真过程中出现异常（如程序卡死、断言失败、外设访问超时）时，系统无法自动分析原因并调整仿真策略。这导致仿真质量的提升高度依赖分析人员的经验，难以适应大规模、多样化的固件测试需求。

1.3 研究内容与主要贡献

为解决上述问题，本文提出一种基于驱动程序替换的物联网设备固件仿真方法。该方法通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型分析驱动函数语义并自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量。本文主要的创新点和贡献有三个：

（1）本研究设计并实现了一个面向嵌入式固件的硬件依赖代码自动识别模块。该模块通过 MMIO 模式识别、数据流追踪与递归归纳等技术，从固件二进制代码中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界，构建可查询的驱动知识库。与现有依赖人工标注或有限规则推断的方法相比，该模块能够自动处理多种嵌入式平台（STM32、NXP 等）和操作系统（FreeRTOS、RT-Thread、裸机）的固件，显著降低了驱动函数定位的人工成本。

（2）本研究设计并实现了一个基于大语言模型的驱动函数分析与替换模块。该模块利用大语言模型的代码理解能力，对识别出的驱动函数进行语义分析，理解其与硬件的交互逻辑，并自动生成可在仿真环境中运行的替代实现。通过设计结构化的提示词模板和函数分类策略，本模块能够处理不同类型的驱动函数（如初始化、数据传输、中断处理等），并通过编译修复循环保障生成代码的可编译性。与现有基于预设返回值的简单 Hook 方法相比，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为。

（3）本研究引入了动态测试反馈闭环机制。当固件在仿真环境中运行时，该机制能够捕获运行时异常（如程序卡死、外设访问超时、断言失败等），分析异常原因，并将分析结果反馈给驱动替换模块以迭代优化替代代码。这种” 生成-测试-反馈-修正” 的闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖。实验结果表明，该反馈机制能够在有限的迭代轮次内显著提高固件在仿真环境中的稳定性和运行覆盖率。

1.4 论文组织结构

本文共分为五章，组织结构如下：

第一章：绪论。介绍研究背景、意义、国内外研究现状以及本文的主要贡献。

第二章：背景知识与相关技术。介绍物联网设备硬件基础、嵌入式操作系统与驱动框架、大模型代码分析技术以及固件仿真与模糊测试基础。

第三章：系统设计与实现。详细描述系统的整体架构设计、各核心模块的设计与实现。

第四章：实验设计与结果分析。介绍实验环境、测试案例、实验结果及分析。

第五章：总结与展望。总结本文工作，讨论局限性，并展望未来研究方向。

2 背景知识与相关技术

本章介绍了物联网设备固件仿真研究涉及的基础知识，包括物联网设备硬件基础、物联网操作系统与驱动框架、大模型代码分析技术以及固件仿真与模糊测试技术，为后续章节的论述奠定基础。

2.1 物联网设备硬件基础

物联网设备通常采用嵌入式微控制器（Microcontroller Unit, MCU）作为核心处理器。常见的 MCU 架构包括 ARM Cortex-M 系列、RISC-V 等。这些处理器具有低功耗、低成本、实时性强等特点，广泛应用于物联网领域。本节重点介绍 ARM Cortex-M 架构的特性以及内存映射 I/O 机制。

2.1.1 ARM Cortex-M 架构特性

ARM Cortex-M 是 ARM 架构处理器核心中低端系列的统称，专为低成本、高能效的微控制器设计，广泛应用于物联网设备。该系列采用 Thumb-2 指令集，在提供 32 位性能的同时保持 16 位代码密度。Cortex-M 架构提供了多项增强实时性和可靠性的特性，包括嵌套向量中断控制器（NVIC）用于中断管理、系统定时器用于提供可编程的周期性中断并支持操作系统的任务调度、以及主栈与进程栈的栈模式切换机制以实现多任务并发和内存保护。

2.1.2 内存映射 I/O

内存映射 I/O（Memory-Mapped I/O, MMIO）是嵌入式系统中外设与处理器通信的主要方式。与端口映射 I/O（Port-Mapped I/O, PMIO）不同，MMIO 不需要专用的 I/O 指令，而是将外设寄存器映射到处理器的统一内存地址空间中。处理器通过普通的内存读写指令（如 LDR/STR）访问特定内存地址，即可完成对外设的控制和数据传输。这种设计简化了编程模型，使得外设访问与内存访问具有相同的语法形式。

2.1.2.1 MMIO 地址空间布局

ARM Cortex-M 架构采用统一的 32 位地址空间，理论上可寻址 4GB 内存。虽然具体的地址映射可能因芯片厂商和型号而有所不同，但 ARM 定义了推荐的内存映射框架，将地址空间按功能划分为若干区域。

代码区位于地址空间的低端，用于存储程序指令和常量数据。该区域支持指令取指操作，通常映射到 Flash 存储器或其他非易失性存储器。SRAM 区紧随其后，用于存储变量、堆栈和动态分配的内存。部分 Cortex-M 处理器支持位带（Bit-Band）操作，允许对单个位进行原子访问，这在嵌入式编程中非常有用。

外设区是 MMIO 的核心区域，所有片上外设的寄存器都被映射到此区域。不同厂商的外设被分配到不同的地址段，例如某厂商的 GPIO 可能位于某一地址范围，而 USART 位于另一地址范围。这种映射使得处理器可以通过标准的内存访问指令来控制外设。

系统区位于地址空间的高端，包含处理器核心相关的调试接口和系统控制寄存器。嵌套向量中断控制器（NVIC）、系统定时器（SysTick）、内存保护单元（MPU）等系统级组件的控制寄存器都位于此区域。此外，系统区还包含厂商特定的系统配置寄存器。

理解 MMIO 地址空间布局对于固件仿真至关重要。仿真器需要正确处理对不同区域的访问：对于代码区和 SRAM 区，可以采用标准的内存模拟；对于外设区和系统区，则需要根据具体外设的行为实现专门的访问处理逻辑。

2.1.2.2 外设寄存器分类

外设寄存器按照功能可分为控制寄存器、状态寄存器和数据寄存器三类。控制寄存器（Control Register）用于配置外设的工作模式和参数，通常是可读写的，固件通过写入特定值来设置外设功能。例如，USART 的控制寄存器 CR1 可以设置波特率、数据位长度、校验位等参数。控制寄存器的写入操作通常不会立即产生硬件响应，而是配置外设在后续操作中的行为。

状态寄存器（Status Register）反映外设的当前工作状态，通常是只读的，固件通

过读取状态寄存器来判断外设是否就绪、是否发生错误等。例如，USART 的状态寄存器 SR 包含 TXE（发送缓冲区空）、RXNE（接收缓冲区非空）等标志位。状态寄存器的值由硬件根据外设实际状态自动更新，这是 MMIO 与普通内存访问的关键区别。

数据寄存器（Data Register）用于数据传输，通常是可读写的，写入操作将数据发送到外设，读取操作从外设接收数据。例如，USART 的数据寄存器 DR 用于发送和接收串口数据。对数据寄存器的访问通常会触发外设执行相应操作（如开始数据传输），并可能改变状态寄存器的值。

2.1.2.3 MMIO 在固件仿真中的挑战

MMIO 机制给固件仿真带来了独特的技术挑战，这些挑战源于硬件外设的复杂性和 MMIO 访问模式的特殊性。

（1）寄存器语义的多样性与不确定性

不同类型的外设具有完全不同的寄存器布局和行为语义。例如，UART 外设的数据寄存器用于串口通信，而 GPIO 外设的数据寄存器用于控制引脚电平。即使是同一类型的外设，不同厂商的实现也可能存在差异，寄存器数量、位域定义、操作时序都可能不同。这种多样性使得难以构建通用的外设仿真模型。更复杂的是，许多嵌入式固件使用厂商自定义的外设，这些外设的文档可能不完整甚至完全缺失，导致仿真器难以准确理解寄存器的语义。

（2）状态寄存器的动态响应问题

状态寄存器的值不是固定的，而是根据外设的内部状态动态变化的。例如，UART 的状态寄存器中的接收缓冲区非空（RXNE）标志位，只有当外设实际接收到数据时才会置位。在仿真环境中，如果没有真实的数据输入，这个标志位应该如何响应？简单的静态返回值无法准确模拟这种行为，而完全模拟外设的状态机又需要深入理解外设的工作原理。这种状态依赖性使得固件可能因为等待某个永远不会出现的状态而无限循环。

（3）寄存器访问的副作用

对 MMIO 寄存器的某些访问会产生副作用，这是 MMIO 与普通内存访问的关键

区别。例如，读取 UART 的数据寄存器不仅返回接收到的字节，还会自动清除接收缓冲区非空标志；写入中断清除寄存器的某些位会清除对应的中断挂起状态。如果仿真器不能正确模拟这些副作用，固件的执行逻辑就会出现错误。识别和理解这些副作用需要仔细分析外设文档，或者通过动态观察真实硬件的行为来推断。

（4）多寄存器操作的原子性与时序约束

许多外设操作涉及多个寄存器的协调访问。例如，配置一个外设可能需要按特定顺序写入多个控制寄存器，如果顺序错误可能导致配置失败或外设状态异常。某些外设对时序有严格要求，如在设置使能位后必须等待一定周期才能访问数据寄存器。这些时序约束在简单的寄存器读写模拟中难以表达，但在真实硬件上是必须遵守的。仿真器需要在保持足够精度的同时，又不能过度限制以避免影响性能。

这些挑战使得传统的内存模拟方法无法直接应用于 MMIO 区域，需要针对不同外设实现专门的仿真逻辑。正是这些复杂性，驱动了基于驱动层替换的仿真方法——通过识别和替换访问 MMIO 的驱动函数，可以在函数调用层面进行抽象，避免直接模拟底层硬件的复杂行为。

这些挑战使得传统的内存模拟方法无法直接应用于 MMIO 区域，需要针对不同外设实现专门的仿真逻辑。这正是驱动层替换方法的核心动机——通过识别和替换访问 MMIO 的驱动函数，避免直接模拟复杂的硬件行为。

2.2 物联网操作系统与驱动

2.2.1 实时操作系统概述

实时操作系统（Real-Time Operating System, RTOS）是物联网设备中广泛采用的管理任务调度和资源分配的软件系统。RTOS 能够及时响应外部事件或数据，并在规定的时间内完成任务，其工作原理是通过一个实时任务调度器来管理系统资源，按照优先级或者时间片来分配 CPU 时间给不同的任务，保证重要或者紧急的任务优先执行。

物联网领域常见的 RTOS 包括 FreeRTOS[?]、Zephyr[?]、RT-Thread[?] 等。FreeRTOS 是 Amazon 开源的实时操作系统，专为微控制器和小型微处理器设计，内核精

简，最小可运行在不到 10KB 的 RAM 上，是物联网领域使用最广泛的 RTOS 之一。Zephyr 是 Linux 基金会管理的开源实时操作系统，支持多种处理器架构，提供更完整的操作系统功能，包括文件系统、网络协议栈、设备驱动模型等，采用模块化设计以平衡功能丰富性和资源占用。RT-Thread 是中国开源社区主导开发的实时操作系统，采用微内核架构，核心组件包括线程调度、线程间通信、内存管理、设备驱动等，并提供了丰富的组件生态。这些 RTOS 都提供了设备驱动框架，为不同类型的外设定义了统一的操作接口，降低了应用开发的复杂度。

2.2.2 硬件抽象层

硬件抽象层（Hardware Abstraction Layer, HAL）是嵌入式软件架构中的关键组成部分，它位于硬件驱动层和上层应用之间，为应用程序提供统一的硬件访问接口。HAL 层的核心目标是屏蔽底层硬件的差异性，使得上层应用代码可以在不同的硬件平台之间移植，而无需修改业务逻辑。

在传统的嵌入式开发中，应用程序需要直接操作寄存器来完成外设控制，这种方式不仅代码可读性差，而且当更换硬件平台时需要大量修改。HAL 层通过定义统一的接口规范，将硬件操作封装为函数调用，显著提高了代码的可维护性和可移植性。例如，不同厂商的 UART 外设寄存器布局可能完全不同，但通过 HAL 层，应用程序可以使用统一的 `HAL_UART_Transmit()` 函数来发送数据，而无需关心底层寄存器的具体实现。

HAL 层的接口设计通常遵循一定的命名和功能规范。以 STM32 的 HAL 库为例，每个外设类型都有一组标准化的接口函数：初始化函数（如 `HAL_UART_Init`）负责配置外设参数和使能时钟；数据传输函数（如 `HAL_UART_Transmit`、`HAL_UART_Receive`）封装了数据的发送和接收逻辑；状态查询函数（如 `HAL_UART_GetState`）用于获取外设的当前工作状态；中断管理函数（如 `HAL_UART_Receive_IT`）用于配置中断模式和注册回调函数。这种标准化的接口设计使得开发者可以快速熟悉不同外设的使用方法。

HAL 层的抽象特性为固件仿真提供了重要基础。由于 HAL 函数封装了具体的硬件操作细节，仿真器可以通过替换 HAL 层函数来实现硬件行为的模拟，而无需深

入理解底层寄存器的语义。这种函数级的替换相比寄存器级的模拟更加高效，也更容易实现。在实际的固件仿真系统中，通过识别和替换 HAL 层的硬件相关函数，可以有效解耦固件逻辑与硬件依赖，使固件能够在没有真实硬件的环境中正常运行。

2.3 大模型代码分析智能体框架

大语言模型（Large Language Model, LLM）在代码理解和生成方面展现出强大的能力，为自动化程序分析提供了新的技术途径。本节重点阐述代码分析工作流范式和基于 LLM 的智能体开发框架，为后续章节的驱动函数分析和替换工作奠定技术背景。

2.3.1 代码分析工作流范式

基于 LLM 智能体框架，可以构建多种代码分析工作流。常见的工作流范式包括：

（1）ReAct 范式：ReAct（Reasoning + Acting）是一种结合推理和行动的工作流范式[?]。在代码分析任务中，LLM 首先进行推理（如" 我需要理解这个函数的功能"），然后采取行动（如" 调用静态分析工具提取函数的调用关系"），观察结果，再进行下一步推理。这种范式使 LLM 能够动态地收集信息，做出决策，逐步完成复杂的分析任务。

（2）Plan-and-Execute 范式：该范式将任务分解为规划和执行两个阶段。在规划阶段，LLM 分析任务要求，生成一个执行计划（如识别硬件相关代码、分析驱动函数语义、生成替代代码、验证编译）。在执行阶段，按照计划依次执行各个步骤。这种范式的优点是结构清晰，易于调试和监控。

（3）Multi-Agent 范式：该范式使用多个专门的智能体协作完成任务[?]。例如，在驱动函数替换任务中，可以设计分析智能体（负责识别驱动函数，分析其语义和依赖关系）、生成智能体（负责生成替代代码，考虑仿真环境的特点）、验证智能体（负责编译验证，检查代码的正确性）、协调智能体（负责协调各智能体的工作，处理冲突）。各智能体通过消息传递或共享状态进行协作，共同完成复杂的代码分析和生成任务。

（4）反思与迭代范式：该范式引入了自我评估和改进机制。在生成初步结果后，

LLM 对结果进行反思（如”这段代码是否正确实现了原函数的功能？”），发现潜在问题，然后进行迭代改进。这种范式特别适用于代码生成任务，能够逐步提高生成代码的质量。在本研究的驱动函数替换模块中，采用了类似的迭代机制：生成代码、编译验证、分析错误、修正代码，直到达到可编译状态。

这些工作流范式为构建可靠的代码分析系统提供了设计模式。在实际应用中，可以根据任务的特点选择合适的范式，或者组合多种范式，以实现最佳效果。

2.3.2 LLM 智能体开发框架

为了更好地利用大语言模型构建复杂的应用系统，研究人员开发了多种智能体（Agent）框架。这些框架提供了结构化的方式来组织 LLM 的调用、管理状态、集成外部工具，使开发者能够构建可靠的、可扩展的 LLM 应用。

2.3.2.1 LangChain 框架

LangChain 是目前最流行的 LLM 应用开发框架之一，提供了丰富的组件和抽象，帮助开发者快速构建 LLM 驱动的应用。LangChain 的核心概念包括模型、提示模板、链、代理、工具和记忆。模型组件封装了与不同 LLM 提供商的交互接口，支持 OpenAI、Anthropic、本地模型等多种后端，开发者可以通过统一的接口调用不同的模型，便于切换和比较。提示模板组件允许开发者定义带变量的提示词模板，在运行时动态填充变量值，使提示词的构造更加灵活和可复用。

链是将多个组件按顺序组合的机制，例如一个代码分析链可能包含读取代码文件、构造提示词、调用 LLM、解析结果、存储分析报告等步骤。通过链的组合，可以构建复杂的处理流程。代理是更高级的抽象，它允许 LLM 自主决定下一步行动。代理根据用户的输入，选择合适的工具（如搜索引擎、代码执行器、数据库查询），执行操作，并根据结果决定后续行动，直到完成任务。LangChain 内置了多种代理类型，如 ReAct 代理[?]、OpenAI Functions 代理等。工具是代理可以调用的外部功能，LangChain 提供了大量内置工具（如网络搜索、Python 代码执行、文件操作等），开发者也可以自定义工具。记忆组件允许应用在多次调用之间保持状态，例如在多轮对话中存储历史对话，使 LLM 能够理解上下文。LangChain 提供了多种记忆实现，

如缓冲记忆、摘要记忆、向量存储记忆等。

2.3.2.2 LangGraph 状态图

LangGraph 是 LangChain 生态中用于构建有状态、可循环的智能体工作流的库。与 LangChain 的链式结构不同，LangGraph 基于状态图（State Graph）的概念，支持更复杂的控制流。LangGraph 的核心是状态（State）对象，它在工作流的各个节点之间传递和更新。状态可以包含任意类型的数据，如对话历史、中间结果、用户输入等。每个节点可以读取和修改状态，实现信息的流动和共享。

节点（Node）是工作流中的处理单元，接收状态作为输入，返回状态的更新。边（Edge）定义了节点之间的转换关系。LangGraph 支持条件边（Conditional Edge），根据状态的值动态选择下一个节点，实现分支逻辑。与简单的链式结构不同，LangGraph 支持循环，允许工作流在满足特定条件前重复执行某些节点。这对于迭代优化、多轮交互等场景非常有用。例如，在代码生成任务中，可以设计一个循环：生成代码、编译验证、如果失败则修复、重新生成，直到代码可编译。LangGraph 还支持检查点（Checkpoint）机制，可以将工作流的状态持久化存储，使得工作流可以暂停和恢复，支持长时间运行的任务。在代码分析场景中，如果 LLM 调用超时或失败，可以从上一个检查点恢复，而不需要重新开始。

2.4 静态分析技术

静态分析技术是软件工程和安全研究的重要基础，旨在通过在不执行程序的情况下，通过分析源代码或二进制代码来推断程序的属性。静态分析的优势在于能够覆盖程序的所有可能执行路径，不受测试用例的限制。常见的静态分析技术包括数据流分析、控制流分析、指针分析、抽象解释等。在固件仿真研究中，静态分析技术用于识别代码中的硬件依赖、追踪数据流、建立函数调用关系，为后续的驱动函数替换提供准确的”硬件依赖画像”。对于固件源码而言，静态分析具有以下优势：首先，源码层面的静态分析能够获取完整的程序信息（类型、符号、注释等），分析精度高于二进制层面的分析；其次，静态分析不需要真实硬件或仿真环境，可以在代

码开发的早期阶段进行；第三，静态分析能够覆盖所有可能的执行路径，不会遗漏任何驱动函数。

2.4.1 CodeQL 静态分析框架

CodeQL 是 GitHub 开发的语义级代码分析平台，支持多种编程语言（包括 C/C++、Java、Python、JavaScript 等）的安全漏洞检测和代码质量分析。CodeQL 的核心思想是将源代码转换为可查询的数据库，然后通过声明式查询语言（QL）进行灵活的分析。与传统基于规则的静态分析工具不同，CodeQL 提供了表达力丰富的查询语言，研究人员可以通过编写查询来查找特定的代码模式、追踪数据流、分析控制依赖等复杂问题。

CodeQL 的工作原理分为两个阶段。第一阶段是数据库构建，CodeQL 对源代码进行解析，生成抽象语法树（AST）、控制流图（CFG）、数据流图（DFG）等多层次的结构化表示，并将这些信息存储在数据库中。第二阶段是查询执行，研究人员使用 QL 语言编写查询，在数据库上执行，获取分析结果。CodeQL 的查询语言基于 Datalog，采用谓词逻辑的表达方式，能够自然地表达”所有存在某种模式的代码片段”或”从源点到汇点的数据流路径”等复杂查询。

CodeQL 在程序分析领域应用广泛，包括安全漏洞检测（如 SQL 注入、缓冲区溢出）、代码质量分析（如死代码检测、复杂度分析）、以及依赖关系分析（如函数调用图、数据流追踪）。本研究利用 CodeQL 的数据流追踪能力，建立 MMIO 访问与函数参数之间的关联关系，为驱动函数识别和语义分析提供支撑。

2.5 固件仿真与模糊测试

固件仿真是物联网安全研究的重要技术手段。由于物联网设备的硬件多样性和封闭性，直接在真实设备上进行安全测试面临诸多困难。固件仿真技术通过软件模拟硬件环境，使研究人员能够在通用计算平台上运行和分析嵌入式固件，从而降低测试成本并提高效率。模糊测试作为一种高效的漏洞挖掘技术，与固件仿真相结合，能够在仿真环境中自动化地发现固件中的安全漏洞。

2.5.1 QEMU 仿真器

QEMU[?] 是目前应用最广泛的开源机器仿真器，支持多种处理器架构和外设仿真，其设计目标是提供高性能、高可移植性的系统仿真能力，使其成为固件仿真研究的重要基础平台。QEMU 支持两种主要的仿真模式：用户模式仿真（User Mode Emulation）允许在一种架构上运行为另一种架构编译的单个程序，适用于分析用户空间程序；系统模式仿真（System Mode Emulation）提供完整的硬件系统仿真，包括处理器、内存、中断控制器、外设等组件，可以运行完整的操作系统或裸机固件，这是固件仿真的主要工作模式。

QEMU 的核心是动态二进制翻译引擎（Tiny Code Generator, TCG），它将目标架构的指令块翻译为中间表示（IR），然后再将 IR 翻译为宿主机指令，这种两级翻译机制使 QEMU 能够支持多种目标架构和宿主架构的组合。为了提高性能，TCG 采用即时编译（JIT）技术，将翻译结果缓存在内存中供后续执行重复使用。

QEMU 提供了丰富的外设模型库，涵盖了常见的通信接口（UART、SPI、I2C）、存储设备（Flash、SD 卡）、网络设备（以太网控制器）等，对于物联网设备中常见的外设，QEMU 通常已经有相应的实现。然而，对于厂商自定义的专用外设，研究人员需要基于 QEMU 的外设框架自行实现仿真模型，这需要深入理解外设的硬件规范，工作量较大。在固件仿真应用中，QEMU 还面临一些挑战：首先是仿真精度与性能的权衡，高精度的仿真需要模拟更多的硬件细节，但会带来显著的性能开销；其次是外设状态的一致性问题，外设的内部状态可能随时间变化，简单的状态机模型难以准确表达这种动态行为；此外，一些物联网设备使用非标准的启动流程和内存映射，需要对 QEMU 进行定制才能正确运行。

2.5.2 模糊测试技术

模糊测试（Fuzzing）是一种自动化软件测试技术，通过向目标程序输入大量随机、半随机或结构化的测试用例，来发现程序中的安全漏洞和缺陷。模糊测试的基本工作流程包括：生成或变异测试用例、将测试用例输入目标程序、监控程序执行状态（如崩溃、断言失败、内存错误等）、记录和分析异常。与传统的手工测试相

比，模糊测试具有高度的自动化能力，能够在短时间内执行数百万次测试，显著提高漏洞发现的效率。根据输入生成策略的不同，模糊测试可分为基于变异的模糊测试（对现有有效输入进行随机变异）和基于生成的模糊测试（根据输入格式规则生成新的测试用例）两大类。

American Fuzzy Lop（AFL）[?] 是最具影响力的覆盖率引导模糊测试工具之一。AFL 的核心思想是通过代码插桩技术记录程序执行过程中的覆盖率信息，利用这些信息指导测试用例的变异和选择，优先保留能够触发新执行路径的测试用例，从而在有限的测试时间内探索更多的程序状态，提高漏洞发现的概率。AFL 的工作流程包含两个核心环节：覆盖率记录和变异策略。在覆盖率记录环节，AFL 通过编译时插桩（修改编译器如 afl-gcc、afl-clang）或运行时插桩（利用 QEMU 的动态翻译功能）在目标程序中插入覆盖率记录代码，记录程序执行过程中经过的基本块和边。在变异策略环节，AFL 维护一个测试用例队列，不断从队列中选择测试用例，对其进行变异，执行变异后的测试用例，并根据覆盖率变化决定是否将新用例加入队列。AFL 的变异策略包括位翻转、字节删除、字节插入、块拼接、算术运算等多种操作，能够产生多样化的测试输入。通过这种迭代式的测试和筛选机制，AFL 能够逐步探索程序的各种执行路径，自动发现潜在的漏洞和缺陷。

在嵌入式固件模糊测试领域，AFL 常与仿真器（如 QEMU）结合使用。研究者在 QEMU 的动态翻译机制基础上实现插桩，使 AFL 能够对在仿真环境中运行的固件进行模糊测试。这种方法不需要修改固件源码，能够直接对二进制固件进行测试，显著提高了测试的自动化程度和可扩展性。

2.6 本章小结

本章介绍了物联网设备固件仿真研究涉及的基础知识，包括物联网设备硬件基础（ARM Cortex-M 架构、MMIO）、物联网操作系统与驱动（RTOS、HAL）、大模型代码分析智能体框架（工作流范式、LLM 框架）、程序分析技术（静态与动态分析、CodeQL）以及固件仿真与模糊测试技术（QEMU、AFL），为后续章节的论述奠定基础。

3 系统设计与实现

本章详细阐述基于驱动程序替换的固件仿真系统的设计与实现。系统采用”静态识别 → 智能替换 → 构建验证 → 动态反馈”的分层架构，通过 CodeQL 静态分析和大语言模型智能体技术，实现固件驱动函数的自动识别、语义分析和代码替换，最终在仿真环境中运行并支持模糊测试。

3.1 整体架构设计

系统分层架构

系统采用分层架构设计，按”静态识别 → 智能替换 → 构建验证 → 动态测试反馈”的链路组织为四个核心模块，如图 3.1所示。每一层模块承担明确的职责，通过标准化的数据接口进行交互，形成完整的处理流水线。

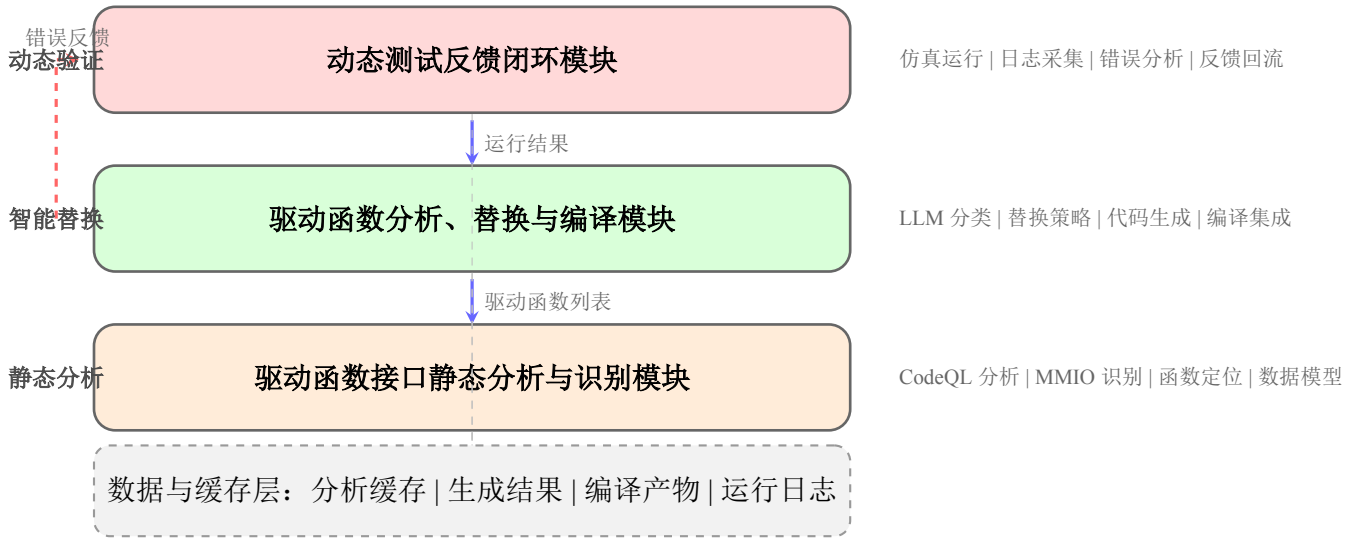

图 3.1 系统整体架构图

架构的顶层是**动态测试反馈闭环模块**，负责在仿真环境中运行替换后的固件，采集运行日志、崩溃信息与覆盖率等指标，并将失败样例与错误信息结构化回流，触发重新分析与再生成。该模块是系统质量保障的关键环节。

中间层是**驱动函数分析、替换与编译模块**，这是系统的核心智能层。该模块基

于上游静态分析结果，使用大语言模型对驱动函数进行类型分类与语义保留分析，生成可替代实现，同时将替代实现集成回工程并驱动构建。该模块采用 LangGraph 智能体框架实现复杂的推理和决策流程。

底层是**驱动函数接口静态分析与识别模块**，这是整个方案的基础支撑层。该模块对固件源码进行静态分析，定位硬件相关代码，抽取驱动函数、MMIO 访问点、依赖数据结构、调用关系等信息，并以统一的数据模型对外提供查询接口。模块基于 CodeQL 静态分析引擎实现，通过预定义的查询规则从固件源码中提取驱动相关信息。

贯穿上述三层的是**数据与缓存层**，统一管理静态分析缓存、生成结果缓存、编译产物与运行日志，支持断点续跑和结果复用。该层确保系统在长时间运行过程中能够保存中间状态，避免重复计算，提高处理效率。

核心模块职责

系统架构中的四个核心模块各司其职，协同工作，共同完成从固件源码到可仿真固件的自动化转换。本节详细阐述各模块的职责定位、设计考量和技术挑战。3.2 3.4 节将分别对四个核心模块进行详细说明。

3.2 驱动函数接口静态分析与识别模块

针对嵌入式固件中驱动函数与硬件交互位置难以自动识别的问题，本研究采用静态分析技术构建驱动函数接口静态分析与识别模块。该模块从固件源码中提取硬件依赖信息，构建固件的” 硬件依赖画像”，为后续的驱动函数替换提供准确的基础数据。

模块的核心职责是建立从源码到硬件依赖信息的映射，识别固件中与硬件交互的函数，确定函数访问硬件的方式（如直接寄存器访问、HAL 库调用或间接句柄操作），并分析函数之间的调用依赖关系。模块采用静态分析技术构建包含语法树、控制流图和数据流图的综合数据库，通过精心设计的查询规则识别 MMIO 结构体实例和字段访问，进而对涉及硬件访问的函数进行语义分类。数据流追踪技术通过污点追踪建立寄存器访问与函数参数之间的关联，为替换模块提供精确的语句定位信息。

模块输出的”硬件依赖画像”包含驱动函数列表、MMIO 访问点、函数调用关系图和数据流信息，这些结构化的分析结果被存储在统一的数据模型中，为下游模块提供可查询的知识库。

3.2.1 MMIO 访问模式识别

MMIO（Memory-Mapped I/O，内存映射 I/O）是一种将硬件外设的寄存器映射到处理器内存地址空间的机制。通过 MMIO，软件可以通过访问内存地址的方式与硬件外设进行交互，而不需要特殊的 I/O 指令。嵌入式厂商通常使用 C 语言结构体来定义 MMIO 寄存器映射，结构体的每个字段对应一个硬件寄存器，字段的偏移量与硬件规范中的寄存器地址偏移保持一致。

MMIO 结构体的典型代码示例如下：

```
// STM32 UART寄存器映射示例
typedef struct {
    __IO uint32_t CR1;     // 控制寄存器1
    __IO uint32_t CR2;     // 控制寄存器2
    __IO uint32_t CR3;     // 控制寄存器3
    __IO uint32_t BRR;     // 波特率寄存器
    __IO uint32_t GTPR;    // 保护时间与预分频寄存器
    __IO uint32_t RTOR;    // 接收超时寄存器
    __IO uint32_t RQR;     // 请求寄存器
    __IO uint32_t ISR;     // 中断与状态寄存器
    __IO uint32_t ICR;     // 中断标志清除寄存器
    __IO uint32_t RDR;     // 接收数据寄存器
    __IO uint32_t TDR;     // 发送数据寄存器
} USART_TypeDef;

// 通过强制类型转换创建MMIO实例
#define USART1_BASE    (0x40011000UL)
USART_TypeDef *USART1 = (USART_TypeDef *)USART1_BASE;
```

定位 MMIO 结构体实例是识别硬件相关代码的关键步骤。本研究总结出三种典型的 MMIO 实例定位模式：

模式一：直接地址强转。这类代码将固定外设基地址强制转换为外设寄存器结构体指针，常见于寄存器定义的头文件中。在代码中呈现为类似 `#define USART1 ((USART_TypeDef *)USART1_BASE)` 的宏定义，后续代码直接对该指针进行字段访问如 `USART1->DR`、`USART1->CR1`。

```
// 驱动源码示例：直接地址强转模式
#define USART1_BASE 0x40011000UL
#define USART1 ((USART_TypeDef *)USART1_BASE)
#define USART2 ((USART_TypeDef *)USART2_BASE)
#define USART3 ((USART_TypeDef *)USART3_BASE)

void uart_init_direct(void) {
```

```
    USART1->CR1 = 0x2000;  // 直接对强转实例进行寄存器访问
    USART1->BRR = 0x0341;
}
```

模式二：间接初始化。这类代码通过句柄（handle）、配置表（descriptor/config table）等间接结构，将外设基地址赋值给某个”寄存器块指针成员”，后续通过该成员完成寄存器访问。在 STM32 HAL 生态中，这类成员常命名为 `Instance`。在代码中呈现为初始化时的赋值语句如 `huart->Instance = COM_USART[COM]`，后续寄存器访问呈现为多级解引用如 `huart->Instance->DR`。在基于配置表的驱动中，也可能呈现为 `handle.Instance = spi_config[i].Instance` 这种从配置表复制的形式。

```
// 驱动源码示例：间接初始化模式（HAL句柄）
UART_HandleTypeDef huart;

void BSP_COM_Init(COM_TypeDef COM) {
    huart.Instance = COM_USART[COM];  // 将外设基地址赋值给Instance成员
    HAL_UART_Init(&huart);
    // 后续访问呈现为多级解引用
    huart.Instance->DR = data;
}

// 驱动源码示例：间接初始化模式（配置表）
typedef struct {
    USART_TypeDef *Instance;
    uint32_t BaudRate;
} SPI_Config;

SPI_Config spi_config[] = {
    {SPI1, 115200},
    {SPI2, 9600}
};

void rt_hw_spi_bus_init(void) {
    for (int i = 0; i < sizeof(spi_config)/sizeof(spi_config[0]); i++) {
        spi_bus_obj[i].handle.Instance = spi_config[i].Instance;
        // 通过配置表中的Instance赋值
    }
}
```

模式三：常量指针参数。这类代码通过函数参数接收 MMIO 结构体指针（寄存器块基址），常见于底层驱动或通用外设操作函数。在代码中呈现为函数定义的形参类型是某个寄存器结构体指针，函数体内部通过该参数访问寄存器字段，如 `void mmio_uart_init(USART_TypeDef *uart) { uart->CR1 = 0x2000; }`。调用者可以将模式一或模式二中的 MMIO 实例作为实参传入，如直接调用 `mmio_uart_init(USART1)` 或传入句柄成员 `mmio_uart_init(huart->Instance)`。

```
// 驱动源码示例：常量指针参数模式
void mmio_uart_init_pointer(USART_TypeDef *uart) {
    // 通过指针访问MMIO寄存器
    uart->CR1 = 0x2000;
```

```
    uart->BRR = 0x0341;
}

// 调用示例
void main(void) {
    // 直接传模式一的强转实例
    mmio_uart_init_pointer(USART1);

    // 传模式二的句柄成员
    mmio_uart_init_pointer(huart.Instance);

    return 0;
}
```

三种定位模式的对比如表 3.1所示。

表 3.1 MMIO 结构体实例定位模式对比

模式	描述	代码呈现
直接地址强转	将固定地址强制转换为外设结构体指针	整数常量 → 结构体指针类型转换
间接初始化	通过配置表将外设基地址赋值给句柄	结构体成员赋值为外设宏
常量指针参数	函数以常量指针形式接收外设基址	参数类型为 MMIO 结构体指针

3.2.2 驱动函数定位

静态分析模块的核心任务是识别固件中的驱动函数，为后续的智能替换提供目标列表。本研究采用简单直接的定位准则：**凡是包含 MMIO 访问的函数，均定义为驱动函数**。这一定位准则的合理性在于：MMIO 访问是固件与硬件交互的最底层方式，包含寄存器读写的函数必然与硬件强相关，这类函数在仿真环境中无法直接执行，需要进行替换处理；而不包含 MMIO 访问的函数通常是纯软件逻辑，如数据处理、协议解析、状态管理等，在仿真环境中可以直接运行，无需替换。

设 F 为固件中的函数集合，M 为 MMIO 结构体实例集合，$A(f, m)$ 为谓词" 函数 f 访问 MMIO 实例 m 的字段"，则驱动函数集合 D 定义为：

$$D = \{f \in F \mid \exists m \in M : A(f, m)\}$$

驱动函数识别的流程如下：首先通过三种定位模式识别代码中的所有 MMIO 结构体实例；然后对每个 MMIO 结构体实例，追踪其字段访问表达式，记录访问的字段名、访问类型（读/写）、所在语句和所在函数；接着对所有包含 MMIO 字段访问的函数进行汇总，形成驱动函数列表；最后分析驱动函数之间的调用关系，构建函数调用图。

3.2.3 分析与识别模块实现

本节阐述分析与识别模块的具体实现技术。模块采用 CodeQL 静态分析引擎作为底层支撑，CodeQL 通过构建包含语法树、控制流图、数据流图的综合数据库，为语义级代码分析提供了强大的查询能力。

3.2.3.1 MMIO 操作定位模块实现

针对三种 MMIO 定位模式，本研究设计了相应的查询规则。查询从 CodeQL 数据库中提取所有符合 MMIO 结构体特征的变量及其初始化表达式。查询的核心设计思想是将”寄存器字段访问”追溯到一个可被 CodeQL 稳定描述的基对象表达式。

```
import cpp
import libraries.MMIOAnalyzer

from
    Function func,
    InterestingMMIOExpr expr
where
    isInterestingMMIOFunction(func) and
    expr.getEnclosingFunction() = func
select expr.toString(),
       expr.getEnclosingFunction().getName(),
       expr.getFile().getAbsolutePath().toString(),
       expr.getLocation().getStartLine(),
       expr.checkExprType(),
       expr.checkEnclosingType()
```

Listing 3.1 MMIO 表达式收集查询示例

上述查询通过 `import libraries.MMIOAnalyzer` 引入 MMIO 分析库，从 CodeQL 数据库中提取所有有趣的 MMIO 表达式。查询的输出包括表达式字符串、所属函数名、文件路径、行号、表达式类型和封闭类型等信息。

对于”间接初始化”和”常量指针参数”模式，查询采用数据流追踪策略，通过追踪赋值语句和函数参数来识别 MMIO 实例。常量基址的追踪利用 CodeQL 提供的数

据流和污点追踪能力，将”常量大值表达式（外设基址）”追踪到后续的变量访问/解引用位置。

```
predicate isMMIOConstantExpr(ConstantExpr src_mmio) {
    exists( DataFlow::Node source, DataFlow::Node sink, Expr varAccess,
             PointerDereferenceExpr deref |
        source.asExpr() = src_mmio and
        isConstantBigValueExpr(src_mmio) and
        sink.asExpr() = varAccess and
        deref.getOperand() = varAccess and
        TaintFlow::flow(source, sink) and
        not sink.asExpr().getFile() instanceof DriverFile
    )
}

predicate isMMIOTracedExpr(Expr e) {
    exists( DataFlow::Node source, DataFlow::Node sink, ConstantExpr src_mmio |
        source.asExpr() = src_mmio and
        isMMIOConstantExpr(src_mmio) and
        sink.asExpr() = e and
        TaintFlow::flow(source, sink) and
        not sink.asExpr().getFile() instanceof DriverFile
    )
}
```

Listing 3.2 常量基址数据流追踪查询示例

上述查询定义了两个谓词：isMMIOConstantExpr 识别常量大值表达式（外设基址），isMMIOTracedExpr 通过污点追踪将源点追踪到汇点。查询的逻辑可形式化表示为：当存在一个常量大值表达式作为数据流源点，经过数据流传播到达某个变量访问或解引用操作时，该操作即为 MMIO 相关的表达式。

```
// 示例：常量基址在代码中的传播
#define USART1_BASE 0x40011000UL
USART_TypeDef *USART1 = (USART_TypeDef *)USART1_BASE;  // 基址强转
huart->Instance = USART1;                               // 赋值给Instance成员
huart->Instance->DR = data;                             // 最终的字段访问点
```

查询通过污点追踪，将上述代码中的 USART1_BASE（源点）追踪到最终的 huart->Instance->DR（汇点），从而识别出整个 MMIO 使用链路。

对于”函数第 0 个参数是指针类型，实参是常量大值表达式”的情况，查询通过函数调用关系和参数类型匹配，定位”把常量基址传给接受结构体指针的驱动函数”的调用点。

```
predicate isMMIOStructure_MK(Struct s){
    exists( FunctionCall call, Expr e, TypedefType typeDef |
        call.getTarget().getParameter(0).getType() instanceof PointerType and
        (
            (call.getTarget().getParameter(0).getType().(PointerType).getBaseType() =
    typeDef and
             typeDef.getBaseType() = s) or
            call.getTarget().getParameter(0).getType().(PointerType).getBaseType() = s
        ) and
```

```
        call.getArgument(0) = e and
        isConstantBigValueExpr(e)
    )
}
```

Listing 3.3 函数参数类型匹配查询示例

上述查询通过参数类型匹配（PointerType）和实参的常量大值表达式检查，识别出 MMIO 结构体类型的函数调用点。查询的逻辑可表示为：当函数调用的第 0 个参数类型为结构体指针，且该参数对应的实参是经过数据流追踪的 MMIO 常量表达式时，该调用点即为候选的 MMIO 使用点。

```
// 示例：常量指针参数传递
void uart_send_data(USART_TypeDef *uart, uint8_t *data, uint16_t len);

// 调用时传递MMIO实例
uart_send_data(USART1, buffer, len);     // 直接传强转实例
uart_send_data(huart->Instance, buffer, len);  // 传Instance成员
```

查询通过参数类型匹配（USART_TypeDef *）和实参的数据流追踪，识别出上述调用点中的 MMIO 参数传递关系。

3.2.3.2 驱动函数定位模块实现

驱动函数定位模块的核心任务是根据前述定位准则，从固件源代码中识别出所有驱动函数，并构建驱动函数调用图。模块采用 CodeQL 静态分析技术，通过定义驱动函数的判定规则来实现自动化识别。

模块的核心判定规则是：**包含 DriverFieldAccess 且不包含 MMIOFieldAccess 的函数被定义为驱动函数**。DriverFieldAccess 指的是通过句柄或间接方式访问 MMIO 寄存器块的字段操作，而 MMIOFieldAccess 指的是直接通过 MMIO 结构体指针访问寄存器的字段操作。这一定义确保了识别出的函数是真正需要替换的驱动层函数，而非底层的硬件抽象层函数。

```
class DriverFunction extends Function
{
    DriverFunction() {
        exists(DriverFieldAccess fa | fa.getEnclosingFunction() = this) and
        not exists(MMIOFieldAccess fa | fa.getEnclosingFunction() = this)
    }
}
```

Listing 3.4 驱动函数判定规则

上述查询定义了 DriverFunction 类，该类继承自 Function，通过检查函数内部是否包含 DriverFieldAccess 且不包含 MMIOFieldAccess 来判定函数是否为驱动函数。

在识别出驱动函数列表后，模块进一步分析驱动函数之间的调用关系，构建函数调用图。调用图以驱动函数为节点，函数调用关系为边，记录调用方、被调用方、调用位置等信息。调用图为后续的智能替换模块提供了函数依赖关系，有助于理解函数的调用上下文和语义关联。

```
from DriverToCall call
select call.toString2(),
       call.getTargetFunction().toString(),
       call.getEnclosingFunction().toString(),
       call.isDirectCall()
```

Listing 3.5 驱动函数调用关系查询

上述查询从 DriverToCall 关系中提取驱动函数之间的调用关系，输出调用边的字符串表示、目标函数名、封装函数名以及调用是否为直接调用等信息。

3.2.3.3 统一数据模型

为便于后续模块使用，静态分析结果被组织为统一的数据模型。数据模型采用 Python 数据类（dataclass）定义，支持序列化和反序列化。

```
# 示例：数据模型定义
@dataclass
class MmioAccessInfo:
    struct_type: str      # 结构体类型，如"USART_TypeDef"
    field_name: str       # 字段名，如"DR"
    access_type: str      # 访问类型：READ/WRITE
    line_number: int      # 行号
    function_name: str    # 所在函数名
    context_code: str     # 上下文代码

@dataclass
class FunctionInfo:
    name: str             # 函数名
    file_path: str        # 文件路径
    line_number: int      # 行号
    signature: str         # 函数签名
    mmio_accesses: List[MmioAccessInfo]  # MMIO访问列表
```

数据模型包括函数信息模型（包含函数名、文件路径、行号、签名、返回类型、参数列表、分类、源码、调用关系等信息）、MMIO 访问点模型（包含结构体类型、字段名、访问类型、行号、所在函数、上下文代码等）和项目分析结果模型（包含项目路径、CodeQL 数据库路径、所有函数、MMIO 访问点、函数调用图、驱动函数列

表、分析时间戳等）。这些数据模型通过 JSON 序列化存储在缓存层，供后续模块查询和使用。

3.3 驱动函数分析、替换与编译模块

驱动函数分析、替换与编译模块是系统的核心智能层，承担着语义理解和代码生成的核心任务。该模块需要在替换驱动函数的同时，确保替换后的代码在仿真环境中能够正常执行，且不破坏原有的业务逻辑。

现有的固件仿真方法主要基于对 MMIO 寄存器的直接模拟，需要在仿真器中为每个外设构建完整的状态机模型。这种方法面临着严重的可扩展性和可靠性问题：不同外设具有完全不同的寄存器布局和行为语义，为每个外设构建准确的状态机模型需要深入了解其硬件规范，且模型构建的复杂度随外设数量的增加而急剧上升；同时，状态机模型的正确性难以验证，模拟错误可能导致固件行为异常；此外，状态机模型的维护成本高昂，当硬件版本更新或引入新外设时，需要重新构建和测试相应的模型。

针对上述问题，本研究采用驱动层函数替换的方法，在函数调用层面进行硬件依赖的解耦，从而避免直接模拟底层复杂的外设状态机和同步机制。该方法的核心思想是：通过识别和替换访问 MMIO 寄存器的驱动函数，将硬件依赖部分替换为与仿真环境兼容的实现，同时保留驱动层的业务逻辑和状态管理，使固件能够在仿真环境中正常运行。

驱动函数替换面临的核心挑战在于如何在去除硬件依赖的同时，保证驱动层状态的完整性和一致性。具体而言，挑战包括：（1）避免固件在等待硬件状态时进入死循环，需要识别并跳过对硬件状态的轮询等待；（2）保证驱动层状态机在软件层面的正常运行，确保初始化函数、配置函数等在上层函数视角下的行为保持一致；（3）维护数据传输路径的正确性，确保串口收发函数、网络传输函数等能够进行正常的数据收发模拟工作，使上层协议栈能够正常运行。

以 UART 驱动为例，替换策略需要满足多方面的需求。首先，需要确保固件不会在等待硬件状态时进入死循环。UART 发送函数通常包含对状态寄存器中 TXE

（发送缓冲区空）标志位的轮询等待，只有当硬件实际发送完数据后该标志位才会置位。在仿真环境中，如果保留这类硬件等待逻辑，固件将无限等待一个永远不会出现的状态，导致程序卡死。因此，替换策略需要识别这些硬件等待逻辑并进行适当的跳过处理。

其次，需要保证初始化函数在软件层面的状态机在上层函数视角下是正常的。UART 初始化函数通常包含对控制寄存器的配置、波特率设置、中断使能等操作。这些操作在外设层面会进入不同的硬件状态，但在驱动层面，初始化函数需要向上层返回正确的状态码，表示初始化成功或失败。替换策略需要确保初始化函数能够向上层正确返回状态，同时维护驱动层内部的状态变量（如发送缓冲区、接收缓冲区、错误标志等），使得上层函数能够正确判断驱动是否可用。

最后，需要保证串口收发函数的正常运行，使底层驱动仍可以进行正常的数据收发模拟工作。UART 发送函数和接收函数通常会检查驱动状态、操作数据缓冲区、触发数据传输等。替换策略需要保留这些逻辑，同时将数据缓冲区操作和触发机制替换为与仿真环境兼容的实现，例如将数据写入虚拟发送缓冲区、从虚拟接收缓冲区读取数据等，使得上层应用和协议栈能够通过驱动函数进行正常的数据交互。

为实现语义保持的驱动函数替换，模块采用大语言模型驱动的智能体架构。与传统基于规则或模板的代码生成方法不同，LLM 具备理解代码语义的能力，能够根据函数的具体实现和上下文生成定制化的替换代码。模块使用 LangGraph 框架构建图式工作流，通过”推理-工具调用-结果汇总”的迭代过程，逐步完成函数分类、策略选择、代码生成和验证等工作。

模块设计了四种替换策略（Skip、Strip、Redirect、Event），每种策略适用于特定类型的驱动函数。策略的选择不是硬编码的规则匹配，而是由 LLM 根据函数的语义特征自主决定。生成替换代码后，模块通过语法检查和代码审查进行质量验证，确保替换代码的可靠性。

编译集成是模块的最后一个环节。模块将替换代码写回源文件，调用构建系统执行编译，并处理可能出现的编译错误。当编译失败时，模块通过 Fixer Agent 进行自动修复，形成”编译-错误解析-修复-重新编译”的闭环，直至编译成功。

3.3.1 驱动函数替换策略设计

为实现语义保持的驱动函数替换，本研究在替换前先对驱动函数进行语义分类，并基于分类结果选择对应替换策略，避免硬件依赖被误删或替换不足导致仿真异常。

3.3.1.1 分类提示词设计

系统使用精心设计的提示词（Prompt）引导 LLM 进行函数分类。提示词的设计直接影响分类的准确性和一致性。本研究采用结构化的提示词模板，包含以下组成部分：

（1）角色定义。明确 LLM 扮演的角色是固件驱动分析专家，建立分析的上下文。

（2）分类标准。详细定义每个类别的判断标准，包括 RECV（读取数据）、SEND（发送数据）、IRQ（中断处理）、INIT（初始化）、CTRL（控制）、POLL（轮询）、DMA（DMA 配置）、NODRIVER（纯软件函数）等。

（3）分析步骤。包括读取函数源码、识别 MMIO 访问、确定主要功能目的、分类并给出理由等步骤。

（4）输出格式。定义 JSON 格式，包含函数名、类别、是否需要替换、替换代码、替换策略、置信度和推理等字段。

3.3.1.2 结构化输出模型

LLM 的分类结果被解析为 Python 数据类，包含函数名、分类、是否需要替换、替换代码、替换策略、置信度和推理等字段。系统使用 LangChain 的结构化输出功能，将 LLM 的 JSON 输出自动解析为该数据类，确保类型安全和字段完整性。

3.3.1.3 替换策略体系

本研究总结出四种通用替换策略，每种策略适用于特定的函数类别和使用场景。策略的选择由 LLM 根据函数的语义特征自动决定。

3.3.1.4 策略一：跳过硬件副作用（Skip）

适用场景：初始化类、控制类函数，寄存器写入对仿真无实际影响。

策略说明：将寄存器写操作替换为无副作用语句，保留函数签名和返回值。对于仿真环境而言，外设初始化和控制操作通常不需要真实执行，因为虚拟外设的状态由仿真器管理。

```c
// 原始代码
void HAL_UART_Enable(UART_HandleTypeDef *huart) {
    huart->Instance->CR1 |= USART_CR1_UE;  // 使能 UART
}

// 替换代码 (Skip 策略)
void HAL_UART_Enable(UART_HandleTypeDef *huart) {
    // Skip: Hardware side effect not needed in emulation
    (void)huart;  // Suppress unused parameter warning
}
```

3.3.1.5 策略二：剥离硬件代码、保留控制逻辑（Strip）

适用场景：发送类函数，需要保留状态更新但移除寄存器操作。

策略说明：删除寄存器访问代码，保留状态管理逻辑（如状态变量更新）。该策略确保驱动状态机正确运行，同时避免对不存在硬件的访问。

```c
// 原始代码
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart,
                                    uint8_t *pData, uint16_t Size) {
    huart->gState = HAL_UART_STATE_BUSY_TX;
    for (int i = 0; i < Size; i++) {
        huart->Instance->DR = pData[i];
        while (!(huart->Instance->SR & USART_SR_TXE));
    }
    huart->gState = HAL_UART_STATE_READY;
    return HAL_OK;
}

// 替换代码 (Strip 策略)
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart,
                                    uint8_t *pData, uint16_t Size) {
    huart->gState = HAL_UART_STATE_BUSY_TX;
    // Strip: Remove register writes, keep state management
    // In emulation, output goes to virtual UART device
    lcmhal_uart_transmit(huart->Instance, pData, Size);
    huart->gState = HAL_UART_STATE_READY;
    return HAL_OK;
}
```

3.3.1.6 策略三：替换为仿真 I/O（Redirect）

适用场景：接收/发送类函数，将操作重定向到仿真平台。

策略说明：使用 LCMHal 虚拟设备接口替代硬件访问。LCMHal 是本系统设计的虚拟外设抽象层，为替换后的驱动代码提供与仿真环境交互的标准接口。

```c
// 原始代码
```

```c
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart,
                                   uint8_t *pData, uint16_t Size,
                                   uint32_t Timeout) {
    for (int i = 0; i < Size; i++) {
        while (!(huart->Instance->SR & USART_SR_RXNE));
        pData[i] = (uint8_t)(huart->Instance->DR & 0xFF);
    }
    return HAL_OK;
}

// 替换代码 (Redirect 策略)
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart,
                                   uint8_t *pData, uint16_t Size,
                                   uint32_t Timeout) {
    // Redirect: Use LCMHal virtual device for I/O
    return lcmhal_uart_receive(huart->Instance, pData, Size, Timeout);
}
```

3.3.1.7 策略四：事件驱动模拟（Event）

适用场景：中断回调类函数，引入事件触发机制。

策略说明：通过事件队列模拟硬件中断触发。在仿真环境中，硬件中断由仿真器生成的事件触发，而非真实的硬件信号。

```c
// 原始代码
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    // 用户实现的回调
    process_received_data(huart->pRxBuffPtr);
}

// 替换代码 (Event 策略)
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    // Event: Trigger callback through event mechanism
    lcmhal_event_t event = {
        .type = LCMHAL_EVENT_UART_RX_COMPLETE,
        .handle = huart
    };
    lcmhal_event_post(&event);
    process_received_data(huart->pRxBuffPtr);
}
```

不同函数类别对应的推荐策略如表 3.2所示。

3.3.2 智能体协作工作流设计

本研究采用多智能体协作的方式，将驱动函数分析、替换和编译的复杂任务分解为多个专门化的智能体，通过 LangGraph 框架构建图式工作流实现智能体之间的协作。智能体协作工作流的核心是将” 分类-替换-验证-编译-修复” 的完整流程自动化，通过智能体之间的信息传递和状态共享，实现端到端的驱动函数替换和编译验证。

表 3.2 替换策略选择指南

函数类别	推荐策略	说明
INIT	Skip	初始化副作用在仿真中不重要
SEND	Strip/Redirect	保留状态或将数据输出到虚拟设备
RECV	Redirect	从虚拟设备读取数据
IRQ	Event	通过事件机制模拟中断
CTRL	Skip	控制操作在仿真中跳过
POLL	Redirect	返回虚拟设备状态
DMA	Redirect	使用虚拟 DMA 接口

3.3.2.1 智能体角色与职责

工作流中包含三个主要智能体：Classifier Agent、VerifyReplacement 和 BuildFixer Agent，每个智能体承担不同的职责，通过工具调用进行协作。

Classifier Agent 是工作流的主导者，负责驱动函数的语义分类和替换代码生成。该智能体接收函数源码、静态分析结果和上下文信息，根据函数的语义特征选择适当的替换策略，生成替换代码。Classifier Agent 采用 LangGraph 框架构建图式工作流，包含推理节点、工具调用节点和结果汇总节点三个核心组件。推理节点由大语言模型驱动，负责进行语义分析和决策；工具调用节点在推理节点需要补充信息时执行工具调用；结果汇总节点对输出进行结构化解析并写入缓存。

VerifyReplacement 是代码审查工具，负责验证替换代码的正确性。该工具接收函数名和替换代码，通过代码审查和可选的编译验证两个层次进行检查。代码审查使用大语言模型扮演代码审查者的角色，根据预定义的规则判断替换代码是否保留了原函数的核心行为，是否引入了潜在的问题。编译验证通过临时应用替换代码并执行编译来验证其语法正确性和链接兼容性。VerifyReplacement 的设计遵循”验证即拒绝”原则，如果代码审查或编译验证失败，直接返回失败结果，拒绝将问题代码持久化到代码库中。

BuildFixer Agent 是编译错误修复工具，负责处理替换代码在编译过程中出现的错误。当 VerifyReplacement 的编译验证失败时，或者 Classifier Agent 生成的替换

代码直接导致编译失败时，BuildFixer Agent 接收编译错误信息和替换代码，分析错误原因并生成修复方案。BuildFixer Agent 具备解析编译器输出的能力，能够提取错误类型、错误位置和错误原因，并根据错误类型选择适当的修复策略。修复成功后，Agent 会将修复后的替换代码写入代码库并执行编译验证，确保修复的有效性。

3.3.2.2 工作流执行过程

智能体协作工作流的执行过程是一个多轮迭代的”分类-验证-修复”循环。工作流从 Classifier Agent 开始，首先对目标函数进行语义分类，确定函数类别和推荐的替换策略。对于需要替换的函数，Classifier Agent 生成替换代码后，会调用 VerifyReplacement 进行代码审查和编译验证。

如果 VerifyReplacement 的验证通过，工作流直接采纳该替换代码，完成当前函数的处理。如果验证失败，工作流根据失败类型采取不同的处理策略。对于代码审查失败的情况，表明替换代码存在明显的语义或逻辑问题，工作流会清空未经验证的替换代码，避免问题代码进入代码库。对于编译验证失败的情况，VerifyReplacement 会返回编译错误信息，工作流会触发 BuildFixer Agent 进行错误修复。

BuildFixer Agent 接收编译错误信息和替换代码后，会使用工具获取函数的上下文信息（包括静态分析结果、其他替换代码等），然后分析错误原因并生成修复方案。修复方案生成后，Agent 会将修复后的替换代码写入代码库，并再次执行编译验证。编译验证采用”临时应用 + 恢复”的机制，确保验证过程不会污染工作树。如果修复后的代码编译成功，工作流采纳该替换代码；如果仍然失败，工作流记录错误信息，并根据重试策略决定是否继续尝试修复。

工作流的最终采纳逻辑采用优先级策略。优先级从高到低依次为：VerifyReplacement 验证通过的代码、BuildFixer Agent 修复成功的代码、未验证的替换代码。这种优先级策略确保只有经过验证的替换代码才会被最终采纳，避免了问题代码进入代码库的风险。对于未经验证的替换代码，如果没有任何验证通过的版本，工作流会直接清空，避免在缺乏验证的情况下采纳不安全的代码。

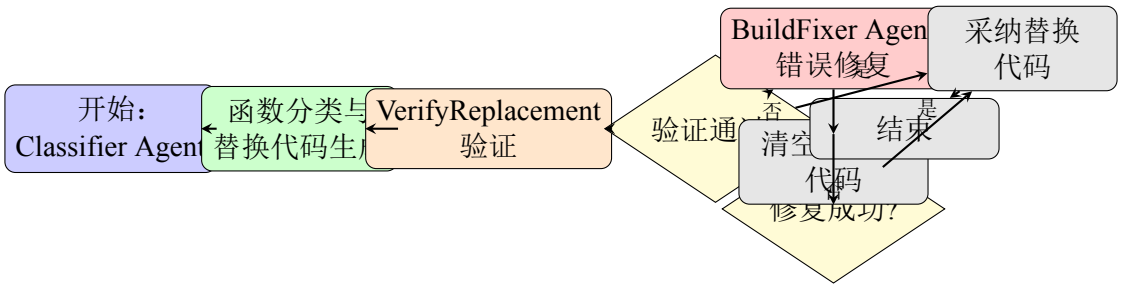

图 3.2 智能体协作工作流图

3.3.2.3 智能体协作的优势

采用多智能体协作的工作流设计具有多方面优势。首先是**职责分离与专业化**，每个智能体专注于特定领域的任务，Classifier Agent 专注于函数语义分析和替换策略选择，VerifyReplacement 专注于代码审查和验证，BuildFixer Agent 专注于编译错误修复，这种职责分离使得每个智能体的实现更加简洁，功能更加聚焦。

其次是**可扩展性与灵活性**，智能体通过工具接口进行交互，新的智能体可以容易地加入工作流，现有的智能体也可以独立升级。当需要支持新的替换策略或新的编译器时，可以通过添加新的工具或智能体来实现，而不需要修改现有的工作流逻辑。

再次是**状态管理与错误恢复**，LangGraph 框架提供了强大的状态管理能力，能够跟踪工作流的执行状态，支持错误恢复和重试机制。当某个智能体执行失败时，工作流能够捕获错误信息，并根据预定义的策略决定是重试还是继续执行其他路径，确保工作流的鲁棒性。

最后是**质量保障的多重验证**，工作流通过代码审查、编译验证和修复验证三重质量保障机制，确保替换代码的正确性和可靠性。代码审查在早期阶段过滤掉明显有问题的代码，编译验证在代码集成前验证其语法和链接正确性，修复验证确保修复方案的有效性，形成完整的质量保障闭环。

3.3.3 分析替换与编译工作流设计

替换代码生成后，系统需要进行质量验证，确保生成的代码能够在编译和运行阶段正常工作。本研究设计了两个层次的验证机制：代码审查和编译测试，形成”轻

量检查 → 完整验证” 的渐进式质量保障体系。

3.3.3.1 代码审查

代码审查在代码生成后立即进行，目的是在编译之前就过滤掉明显有问题的替换代码，避免浪费编译时间和资源。

本研究采用 LLM 驱动的代码审查方法。该方法的核心思想是：将原函数代码和替换代码一起提交给 LLM，让 LLM 扮演代码审查者的角色，根据预定义的规则进行判断。LLM 具备理解代码语义的能力，能够识别替换代码是否保留了原函数的核心行为，是否引入了潜在的问题。

代码审查的规则通过提示词（Prompt）定义，主要包括以下方面：

签名一致性规则。要求替换函数的名称、返回类型、参数列表必须与原函数完全一致，这是替换代码能够正确集成回工程的基本前提。

语义保持性规则。要求替换代码应保留原函数的核心业务逻辑，如状态变量更新、错误处理流程等，只替换与硬件交互的部分。

代码合理性规则。要求替换代码不应引入明显的问题，如未声明变量、无限循环、内存泄漏等。

调用关系保持规则。如果原函数调用了某些核心函数（如回调函数、状态更新函数），替换代码应在适当的位置保留这些调用。

代码审查的实现采用 LLM 结构化输出技术。系统定义了 `CodeReviewResult` 数据类作为输出格式，包含 `passed`（是否通过检查）和 `reason`（未通过的原因）两个字段。通过 LangChain 的结构化输出功能，强制 LLM 按照该格式输出，确保结果可以被程序正确解析。

3.3.3.2 编译测试

代码审查通过后，系统进行编译测试。编译测试是替换代码质量验证的关键环节，它将替换代码集成回工程，调用构建系统执行完整的编译链接流程，验证替换代码的语法正确性和链接兼容性。

编译测试的流程包括四个阶段：

第一阶段：代码集成。系统将替换代码写入源文件，替换原函数的定义。

第二阶段：构建执行。系统调用项目的构建脚本（如 CMake、Makefile）执行编译。

第三阶段：错误解析。如果编译失败，系统需要解析编译器的错误输出，提取错误类型、错误位置和错误原因。

第四阶段：错误修复。系统根据错误信息触发 Fixer Agent 进行自动修复，修复后重新编译，直到成功或达到重试上限。

编译集成是驱动函数分析、替换与编译模块的最后一个环节，负责将替换后的代码集成回工程并执行编译。这一环节由 Builder Agent 负责，它是系统中专门处理构建任务的智能体。

Builder Agent 采用 LangGraph 框架实现，具有状态管理和循环执行的能力。Agent 的工作流程是一个迭代过程：首先尝试编译，如果失败则分析错误并尝试修复，修复后重新编译，直到成功或达到重试上限。

Builder Agent 的核心能力包括：

（1）构建环境配置。读取项目的构建配置（如 CMakeLists.txt、Makefile），了解项目的构建方式和依赖关系。

（2）编译错误分析。当编译失败时，从编译器的错误输出中提取关键信息，判断错误的类型和严重程度。

（3）修复触发。当编译失败时，触发 Fixer Agent 进行错误修复。

（4）产物验证。编译成功后验证 ELF 文件的完整性。

图 3.3 编译错误修复闭环

3.3.4 具体实现

在具体实现层面，本系统采用 LangChain 作为整体智能体框架，组织“推理-工具调用-结果汇总”的迭代过程，并以 LangGraph 构建有状态、可循环的工作流，从

而实现替换策略生成、质量验证与构建集成的协同执行。

3.3.4.1 LangGraph 工作流框架

系统采用 LangGraph 框架构建图式工作流。LangGraph 是 LangChain 生态中用于构建有状态、可循环的智能体工作流的库，支持复杂的状态管理和条件分支。

智能体工作流的核心包含三个逻辑节点：**推理与决策节点**（由大语言模型驱动）、**工具调用节点**（当推理节点需要补充信息时请求调用工具）、**结果汇总节点**（对输出进行结构化解析并写入缓存）。

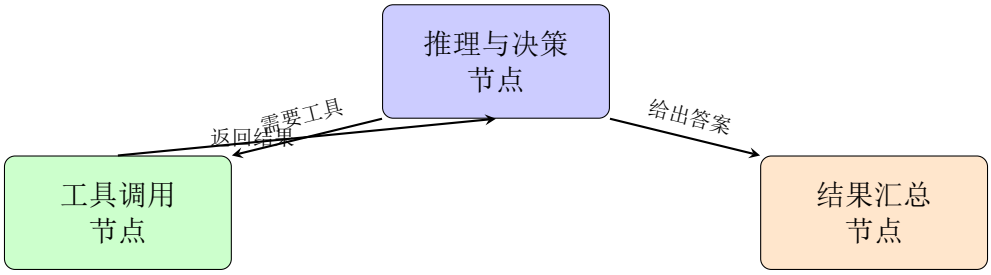

图 3.4 智能体工作流架构图

3.3.4.2 状态机设计

LangGraph 的核心是状态（State）对象，它在工作流的各个节点之间传递和更新。状态模型需要维护智能体执行过程中的关键信息。

状态转移逻辑控制智能体的执行流程。智能体从 START 状态开始，首先进入推理节点，LLM 根据当前状态决定下一步行动。如果 LLM 请求调用工具（如查询函数源码、获取调用关系），则转移到工具节点，工具执行完毕后返回推理节点继续处理。如果 LLM 认为已经获得足够信息并给出最终答案，则转移到结果节点，对输出进行结构化解析后进入 END 状态。如果在执行过程中发生错误，且重试次数未超过上限，则返回推理节点重新尝试；否则进入错误处理流程。

智能体状态机转移图（示意）

图 3.5 智能体状态机转移图

3.4 动态测试反馈闭环模块

动态测试反馈闭环模块是系统的”验证官”，负责在仿真环境中检验替换效果，并将发现的问题反馈给上游模块。该模块体现了软件工程中”测试驱动”的思想：静态分析和代码替换的正确性最终需要通过实际运行来验证。

3.4.1 动态执行环境

动态阶段将替换后的固件部署到仿真平台执行，统一输出运行日志、崩溃信息、执行统计和覆盖率指标。动态测试是验证替换效果的关键环节，也是反馈闭环的数据来源。本系统的动态执行环境基于 Unicorn 模拟引擎构建，这是一个专门为适配 LCMHal 而设计的嵌入式固件模拟执行平台。

仿真平台的系统架构采用分层设计，包括配置层、模拟层、日志层、模型层、处理层和工具层。配置层负责解析和管理固件配置，包括内存映射、符号表、处理器配置等。模拟层基于 Unicorn 实现 ARM 指令集模拟，包括内存管理、外设模拟和中断处理。日志层提供多种日志输出通道，包括调试日志、函数调用日志和 LCMHal 专用日志。模型层实现了循环检测队列和函数调用跟踪器，用于优化执行效率和收集执行信息。处理层实现了各种 HAL 函数的处理器，特别是针对 LCMHal 的 `HAL_BE_*` 系列函数。工具层提供了配置生成、符号提取等辅助功能。

仿真平台支持两种运行模式：模拟执行模式和模糊测试模式。模拟执行模式用于验证固件的基本功能和执行流程，系统会完整执行固件的初始化流程和主循环，记录执行轨迹和函数调用关系。模糊测试模式用于安全测试和漏洞挖掘，系统会在关键点设置 fork 点，实现高效的模糊测试，支持覆盖率引导的测试用例生成。

3.4.1.1 内存映射与执行初始化

仿真平台实现了精确的内存映射管理，针对 LCMHal 的需求进行了专门优化。系统定义了受保护的内存类型，包括 flash（代码存储区）、ram（随机访问存储器）和 peripheral_ram（外设 RAM），这些区域不应相互合并，确保模拟的精确性。

执行初始化阶段，仿真平台实现了智能入口点选择机制。系统优先使用 Reset_Handler 作为入口点，通过解析 ELF 符号表查找 Reset_Handler 的地址，确保从

正确的启动代码开始执行。此外，系统还添加了 VFP（浮点处理单元）支持，通过设置 FPEXC.EN 和 CPACR 寄存器，确保固件中的浮点指令能够正确执行。

3.4.1.2 循环检测与执行优化

针对嵌入式固件初始化代码中常见的大量循环，仿真平台实现了智能循环检测机制，防止初始化代码被误判为死循环，提高模拟执行效率。

循环检测采用多种策略相结合的方式：

（1）跳过前 N 个唯一基本块策略。在固件初始化阶段，系统跳过对前 N 个唯一基本块的循环检测，因为这些基本块通常包含必要的初始化代码，不会构成死循环。

（2）循环白名单机制。系统维护一个函数白名单，对于白名单中的函数，跳过循环检测。这些函数通常是已知的轮询等待函数或长时间运行的初始化函数，如 `HAL_Init`、`SystemClock_Config` 等。

（3）最大循环次数限制。对于不在白名单中的函数，系统记录每个基本块的执行次数，当执行次数超过配置的阈值（默认为 5000 次）时，判定为异常循环并终止执行。

3.4.2 LCMHal 仿真接口

LCMHal（Lightweight Communication and Hardware Abstraction Layer）是本系统设计的虚拟外设接口层，为替换后的驱动代码提供仿真 I/O 能力。LCMHal 的实现分为弱符号桩函数、符号表映射和 Python 处理器三个层次。

仿真平台通过配置文件定义函数名到 Python 处理器的映射关系。对于 `HAL_BE_In`、`HAL_BE_Out`、`HAL_BE_ENET_ReadFrame` 等函数，系统分别实现了对应的 Python 处理器，这些处理器通过 Unicorn 模拟器的寄存器访问接口和内存写入接口模拟外设行为。

3.4.3 MMIO 函数跟踪与动态反馈

为了支持 LCMHal 的动态分析需求，仿真平台实现了 MMIO 函数执行信息的跟踪和反馈机制。配置文件中的 `mmio_funcs` 列表定义了需要跟踪的 MMIO 函数，当模拟执行到这些函数时，系统会记录调用信息并写入 LCMHal 专用日志文件。

除了 MMIO 函数跟踪，系统还会反馈异常循环检测信息和错误上下文信息。当检测到异常循环时，系统记录当前函数名、调用者函数名和循环位置，写入日志文件。当发生内存访问错误或其他异常时，系统记录错误地址、调用栈和错误指令的反汇编代码，为问题定位提供完整的上下文。

3.4.4 Emulator Runner Agent

Emulator Runner Agent 是系统中负责执行仿真的智能体，它将编译后的固件部署到 Unicorn 引擎，监控运行状态，采集日志数据，并在出现问题时生成反馈信息。

Agent 的工作流程包括四个阶段：

仿真环境准备阶段。根据目标固件的硬件平台配置 CPU 架构模式和内存映射，准备仿真环境。

固件加载阶段。将编译生成的 ELF 文件加载到仿真器的内存中，设置程序计数器（PC）和栈指针（SP）到正确的初始值。

执行监控阶段。启动仿真器运行固件，同时实时监控运行状态，采集日志数据。监控的内容包括标准输出和标准错误、异常信号、程序退出码等。

结果分析阶段。当仿真结束（正常退出或异常终止）后，Agent 对采集的日志和状态信息进行分析，判断运行是否成功，如果失败则提取失败的原因和位置。

Agent 的实现基于 LangGraph 框架，具有状态管理和循环执行的能力。Agent 的决策逻辑由 LLM 驱动，LLM 根据当前的运行状态和日志信息，决定是否需要继续运行、是否需要调整仿真参数、是否需要生成反馈等。

3.4.5 反馈信息结构化

仿真运行产生的原始日志通常是大量的、非结构化的文本信息，包含大量的调试输出、警告信息、错误堆栈等。直接将这些原始信息反馈给上游模块效果不佳，因为原始日志中包含大量无关信息，关键信息难以提取。因此，系统需要对原始日志进行结构化处理，提取关键信息，形成简洁、准确的反馈摘要。

反馈摘要的数据结构设计考虑了信息完整性和可用性的平衡。摘要包含：

失败类型。包括编译失败、链接失败、运行崩溃、运行超时等。

关键证据。包括错误日志的关键片段，如异常地址、崩溃堆栈、断言失败信息。

关联函数。与当前失败相关的替换函数列表，通过调用栈分析确定。

复现输入。如果是在模糊测试场景中，记录触发失败的输入样本。

修复建议。基于错误类型和上下文生成的修复提示。

3.4.6 迭代优化机制

动态测试反馈闭环的核心是迭代优化机制。当仿真运行发现问题时，系统启动一个迭代优化流程，通过多轮的”问题分析 → 代码修复 → 重新测试”，逐步解决运行时问题。

迭代优化由 Fixer Agent 驱动。当 Emulator Runner Agent 检测到运行失败时，它将反馈摘要传递给 Fixer Agent。Fixer Agent 是系统中专门处理运行时修复的智能体，它接收反馈摘要、出错的替换代码、原函数代码等上下文信息，然后生成修复方案。

Fixer Agent 的修复策略包括：调整等待条件的实现（如添加超时机制、修改轮询条件）、修正状态管理逻辑（如补充遗漏的状态更新）、调整替换策略（如从 Strip 策略改为 Redirect 策略）、重新生成替换代码等。修复代码生成后，系统将其应用到源文件，重新执行编译和仿真，验证修复效果。

通过这种迭代优化机制，系统能够自动解决大部分运行时问题，显著减少人工干预的需求。系统记录每次迭代的详细信息（包括失败现象、修复方案、验证结果），为问题追溯和系统优化提供了宝贵的数据。

3.4.7 完整工作流

系统实现”静态识别 → 智能替换 → 构建验证 → 动态反馈”的完整闭环，如图 3.6所示。整个处理流程从固件源码出发，经静态分析产生”硬件依赖画像”，随后驱动替换模块在该画像约束下生成替代代码并集成构建，最后由动态执行产生反馈数据回流。每个处理阶段的输出都成为下一阶段的输入，形成有向且可追溯的处理链路。

系统以”任务”（Task）为单位组织处理流程，每个任务对应一个固件工程、一个构建配置与一组待替换函数列表。任务的状态转移如图 3.7所示，从初始的”未分

析”状态，依次经过静态分析、代码生成、编译构建，最终进入”已运行”状态表示任务成功完成。如果在运行阶段检测到问题（如运行崩溃、功能异常），任务进入”需重试”状态，系统会根据反馈信息重新分析问题函数，生成新的替换代码，然后重新进入生成状态，开始新一轮的构建和测试。这个闭环机制确保系统能够通过迭代优化不断提高替换质量。

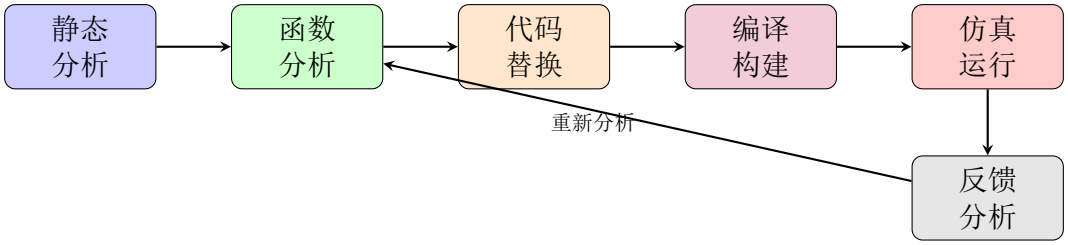

图 3.6 完整工作流程图

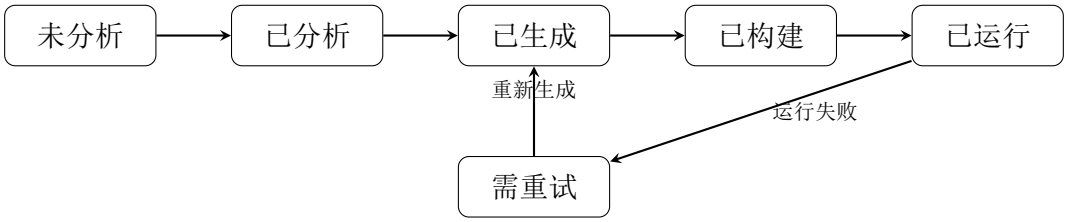

图 3.7 任务状态机

3.5 本章小结

本章详细阐述了基于驱动程序替换的固件仿真系统的设计与实现。系统采用分层模块化架构，实现了从静态识别到动态反馈的完整闭环。

系统的主要贡献包括：

（1）静态分析模块。基于 CodeQL 实现驱动函数和 MMIO 访问的自动识别，支持三种 MMIO 定位模式和八类函数语义分类，为智能替换提供准确的”硬件依赖画像”。

（2）LLM 驱动的替换模块。采用 LangGraph 智能体架构，实现驱动函数的自动分类和代码替换。系统设计了四种替换策略（Skip、Strip、Redirect、Event），支持不

同类型驱动函数的语义保持替换。

（3）动态测试反馈闭环。在 Unicorn 引擎中运行固件，通过 LCMHal 虚拟外设接口实现仿真 I/O。系统建立了完整的反馈机制，能够自动检测运行问题并触发修复流程。

该设计实现了”静态识别 → 智能替换 → 构建验证 → 动态反馈”的完整闭环，为后续实验验证奠定了坚实基础。

4 实验设计与结果分析

本章通过系统性的实验验证本文提出的基于驱动程序替换的固件仿真方法的有效性。实验从多个维度对方法进行评估，包括静态分析模块的识别准确率、运行正确性、方法对比以及应用验证。

4.1 实验设计

为了全面评估本文方法的有效性和实用性，本节介绍实验的整体设计，包括实验目标、测试固件的选择、实验环境的搭建以及评估指标的定义。

4.1.1 实验目标

本章实验旨在验证以下研究问题：

RQ1（识别准确性）：静态分析模块能否准确识别固件中的硬件依赖代码？识别的准确率和召回率如何？该指标反映了静态分析模块对固件硬件依赖的感知能力，是后续智能替换的基础。如果静态分析模块无法准确识别硬件依赖代码，将导致驱动函数替换的不完整或不准确，影响整个方法的自动化效果。

RQ2（运行正确性）：替换后的固件能否在仿真环境中正确运行？能否正常启动、执行主要功能、保持语义一致性等？该指标是验证系统核心能力的关键，即”替换后的固件能否正常运行”。运行正确性包括编译正确性、启动正确性、功能完整性和运行稳定性四个方面。如果替换后的固件无法在仿真环境中正确运行，那么本文方法就无法支持后续的安全测试和漏洞挖掘任务。

RQ3（方法对比）：与现有的固件仿真方法相比，本文方法在效果和效率上有何优势？该指标用于评估本文方法的创新点和实用价值。与 Halucinator 和 Para-rehosting 等现有方法相比，本文方法通过 LLM 驱动的智能替换机制，在自动化程度、准备时间和可扩展性方面应该具有显著优势。

RQ4（应用验证）：本文方法能否有效支持后续的模糊测试和漏洞挖掘任务？该指标用于验证本文方法的实际应用价值。如果替换后的固件能够在仿真环境中稳定

运行并支持模糊测试，那么本文方法就能够为物联网设备固件的安全测试提供有效的技术途径。

4.1.2 测试固件选择

为了验证方法的通用性和有效性，本研究选择了多种典型的物联网设备固件作为测试案例，涵盖不同的芯片平台、操作系统、驱动复杂度和应用场景。

（1）芯片平台选择

测试固件覆盖了物联网设备中常见的两类主流 MCU 平台：STM32F4 系列（ARM Cortex-M4）和 NXP i.MX RT 系列（ARM Cortex-M7）。

（2）操作系统选择

测试固件覆盖了三种典型的嵌入式软件架构：FreeRTOS、RT-Thread 和裸机。FreeRTOS 是全球使用最广泛的开源实时操作系统，具有内核精简、易于移植的特点，支持任务管理、队列、信号量等功能。RT-Thread 是国产开源实时操作系统，提供完整的设备驱动框架和丰富的软件包生态，在国内物联网领域应用广泛。裸机是指无操作系统支持的固件，所有驱动和应用代码直接运行在硬件上，常见于资源极度受限的物联网设备。

（3）外设类型选择

测试固件涵盖了物联网设备中常见的四类外设驱动，详细信息如表 4.1所示。

表 4.1 外设类型分类

类别	外设类型
通信外设	UART（串口通信）、I2C（总线通信）、SPI（串行外设接口）、ETH（以太网控制器）
存储外设	Flash（闪存控制器）、SDIO（SD 卡接口）
定时器外设	TIM（通用定时器）、RTC（实时时钟）、WDT（看门狗定时器）
其他外设	BLE（蓝牙）、FatFS（文件系统）

（4）测试固件构建

本研究测试固件来源于官方示例与开源项目，从 STM32CubeMX 和 NXP MCUXpresso SDK 提取的官方示例程序代表规范的驱动代码编写风格，而从 GitHub

和 Gitee 收集的开源的物联网设备固件项目涵盖智能照明、环境监测、工业控制等多种应用场景。这种来源选择确保了测试固件的多样性和代表性。

经过筛选和整理，最终构建了包含 20 个固件的测试集，详细信息如表 4.2所示。

表 4.2 测试固件详细信息

编号	MCU 型号	OS	外设/驱动库	Demo 名称
F01	STM32F407	FreeRTOS	UART, I2C, STM32 HAL	NXP_FATFS_BareMetal
F02	STM32F407	FreeRTOS	SPI, ETH, LwIP, FreeRTOS	NXP_FATFS_FreeRTOS
F03	STM32F407	RT-Thread	UART, ADC, RT-Thread	NXP_I2C_BareMetal
F04	STM32F407	裸机	UART, GPIO, STM32 HAL	NXP_I2C_FreeRTOS
F05	STM32F407	裸机	I2C, TIM, STM32 HAL	NXP_LwIP_BareMetal
F06	STM32F429	FreeRTOS	UART, ETH, LwIP, FreeRTOS	NXP_LwIP_FreeRTOS
F07	STM32F429	FreeRTOS	SPI, Flash, STM32 HAL	NXP_UART_BareMetal
F08	STM32F429	RT-Thread	I2C, ADC, RT-Thread	NXP_UART_FreeRTOS
F09	STM32F429	裸机	UART, DAC, STM32 HAL	imxrt1052-nxp-evk
F10	STM32F429	裸机	TIM, WDT, STM32 HAL	imxrt1060-nxp-evk
F11	RT1052	FreeRTOS	UART, ETH, LwIP, FreeRTOS	BLE_HeartRateFreeRTOS
F12	RT1052	FreeRTOS	SPI, SDIO, NXP HAL	FatFs_USBDisk_RTOS
F13	RT1052	RT-Thread	I2C, RTC, RT-Thread	LwIP_HTTP_Server_Raw
F14	RT1052	RT-Thread	UART, Flash, RT-Thread	LwIP_HTTP_Server_SocketRTOS
F15	RT1052	裸机	GPIO, ADC, NXP HAL	LwIP_HTTP_Server_Socket_RTOS
F16	RT1052	裸机	UART, TIM, NXP HAL	LwIP_HTTP_Server_Socket_RTOS
F17	STM32F407	FreeRTOS	多外设	LwIP_StreamingServer
F18	STM32F429	RT-Thread	多外设	UART_Hyperterminal_IT
F19	RT1052	FreeRTOS	多外设	-
F20	RT1052	RT-Thread	多外设	-

4.1.3 实验环境

实验在以下软硬件环境中进行：

（1）硬件环境

（2）软件环境

表 4.3 硬件环境配置

配置项	规格
主机	Intel Xeon Platinum 8350C 处理器（128 核心），503GB 内存，869GB SSD

表 4.4 软件环境配置

配置项	规格
操作系统	Ubuntu 22.04 LTS
编译工具链	ARM GCC 10.3.1、CMake 3.21.4
仿真环境	Unicorn 2.0.2、UnicornAFL（基于 Unicorn 的模糊测试扩展）
模糊测试	AFL++ 4.08c+（基于 v4.08c 的开发版本）

（3）框架集成

本文方法的各个模块通过 Python 脚本进行集成，形成完整的自动化工具链。静态分析模块使用 CodeQL 查询驱动函数，LLM 替换模块通过 API 调用大语言模型，编译修复模块使用 ARM GCC 进行编译验证，动态反馈模块通过 Unicorn 监控固件运行状态，仿真执行基于 Unicorn 引擎实现，支持 ARM Cortex-M 架构的指令级仿真。

4.1.4 评估指标

本研究定义以下评估指标：

（1）静态分析模块指标

- **识别准确率（Precision）**：正确识别的驱动函数数量 / 识别出的驱动函数总数
- **识别召回率（Recall）**：正确识别的驱动函数数量 / 实际存在的驱动函数总数
- **F1 分数**：准确率和召回率的调和平均数

（2）运行正确性指标

- **编译正确性**：最终编译成功率、平均修复轮次
- **启动正确性**：启动成功率、关键节点匹配率

- **功能完整性**：功能验证结果、关键节点覆盖度
- **运行稳定性**：崩溃次数、运行时间、稳定性评价
- **通用性**：驱动库种类、平台数量、操作系统数量

（3）方法对比指标

- **准备时间**：替换层编写/移植、端到端自动化执行时间
- **自动化程度**：手工编写 vs 完全自动化
- **运行正确性**：编译成功率、启动成功率、运行稳定性
- **可扩展性**：人力资源依赖、源代码依赖、冷启动成本

（4）模糊测试指标

- **基本块覆盖率**：被测试覆盖的代码块占所有代码块的比例
- **执行速度**：每秒执行的测试用例数量（exec/s）

4.2 静态分析模块评估

本节评估静态分析模块在识别固件中硬件依赖代码方面的准确性和效率。

4.2.1 基准数据集构建

为了评估静态分析模块的识别准确性，需要构建包含真实驱动函数标注的基准数据集。对于测试集中的每个固件，通过以下方式标注驱动函数：

- 对于官方示例与开源项目，通过阅读源码和文档，手工标注所有与硬件交互的驱动函数
- 对于开源项目，通过检查代码中的寄存器访问和 HAL 函数调用，确定驱动函数

- 标注信息包括：函数名称、所属外设类型、是否直接访问寄存器、调用关系等

经过标注，20 个固件共包含 342 个驱动函数，平均每个固件 17.1 个驱动函数。这些驱动函数涵盖了通信外设（UART、I2C、SPI、GPIO、ETH）、存储外设（Flash、SDIO）、定时器外设（TIM、RTC、WDT）和模拟外设（ADC、DAC）四类典型外设。

4.2.2 识别准确性评估

使用静态分析模块对测试集中的所有固件进行分析，统计识别出的驱动函数，并与基准数据集进行对比。实验结果如表 4.5所示。

表 4.5 静态分析模块识别准确性

外设类型	准确率	召回率	F1 分数
UART	95.2%	93.8%	94.5%
I2C	92.7%	90.4%	91.5%
SPI	93.1%	91.2%	92.1%
GPIO	96.5%	95.2%	95.8%
TIM/RTC/WDT	89.6%	87.5%	88.5%
ETH	88.2%	85.6%	86.9%
Flash/SDIO	94.8%	92.7%	93.7%
平均	**92.5%**	**90.4%**	**91.4%**

实验结果表明，静态分析模块在识别驱动函数方面达到了较高的准确率。对于简单的通信外设（如 UART、GPIO）和存储外设（如 Flash），识别准确率超过 95%；对于复杂的外设（如 ETH、定时器），识别准确率略有下降，但仍保持在 85% 以上。

4.2.3 错误分析

对识别错误的案例进行分析，发现主要原因包括：

（1）误报（False Positive）：约 7.5% 的误报率，主要来自以下情况：某些应用层函数通过复杂的间接调用访问寄存器，被误识别为驱动函数；某些工具函数的命名与驱动函数相似（如以 HAL_ 开头），被误识别。

（2）漏报（False Negative）：约 9.6% 的漏报率，主要来自以下情况：某些驱动

函数通过函数指针调用，静态分析难以追踪；某些自定义的驱动实现没有使用标准的 HAL 接口，模式特征不明显。

4.3 运行正确性评估

本节评估替换后的固件在仿真环境中的运行正确性。运行正确性是验证系统核心能力的关键指标，即”替换后的固件能否正常运行”。

为了全面评估替换后固件的运行正确性，本研究采用基于关键节点检查的验证方法。该方法的核心思想是：通过对各个代表性 Demo 进行模拟仿真运行，然后通过检查关键节点（函数、事件、中断）是否被执行到来确认固件的执行是否正常。与传统的基于硬件的测试方法不同，本方法关注固件在仿真环境中的执行轨迹和关键事件，而非具体的硬件状态。这种方法的优势在于：适合无硬件场景的验证、可以快速验证多个固件、验证结果可追溯可复现、适合大规模自动化测试。

4.3.1 验证方法

本节介绍验证方法的设计思路、验证流程、测试用例选择和评估指标。

（1）验证方法的设计思路

本研究提出的验证方法基于以下设计思路：

第一，**关键节点选择**。关键节点是指在固件执行过程中能够反映其功能和状态的重要事件，如系统启动、功能触发、中断处理等。关键节点的选择需要根据固件的功能和特点进行定制，对于不同类型的固件，关键节点可能不同。例如，对于使用 RTOS 的固件，RTOS 内核启动和任务调度是关键节点；对于网络功能的固件，网络协议栈初始化和数据收发是关键节点；对于文件系统功能的固件，文件系统初始化和文件操作是关键节点。

第二，**执行轨迹分析**。通过在仿真环境中运行固件，记录其执行轨迹，然后分析关键节点是否被执行到来。执行轨迹分析可以提供详细的执行信息，包括函数调用关系、中断触发顺序、事件发生时间等。这些信息对于验证固件的执行是否正常具有重要价值。

第三，**功能确认**。通过检查关键节点是否被执行到来，确认固件的主要功能是